Aspis Spend: Transparent Shielded-Spend Verification and Atomic State Update from a Pre-Uploaded Proof Account on Solana

D. Barker

15 July 2026

Abstract

This paper presents Aspis Spend, a transparent argument for a depth-20, one-input/one-output shielded-spend relation together with a Solana program that verifies the argument and atomically records the corresponding nullifier and pool-state transition. The accepting instruction consumes a finalized, pre-uploaded proof account; proof-account creation, chunk upload, and finalization are separate setup transactions. The polynomial-commitment layer is a custom circle-domain, WHIR-style multilinear protocol at rate 1/512, with 18 sampled query fibers and batch-work parameter $g = 37$. Under the stated finite-parameter and boundary-state-restoration premises, the frozen event ledger and its 32-boundary Fiat–Shamir reduction give a conservative 100.161-bit floor for work-normalized false acceptance per random-oracle query. Under the stated affine-image premise, a witness-free simulator gives 104.024 bits for a real view versus simulation and 103.024 bits for pairwise witness-indistinguishability in the declared programmable-SHA-256 random-oracle, fixed Proof-or-Abort view; this is a conditional computational zero-knowledge statement for that declared view. The pinned implementation artifact contains a 65,407-byte proof and a 924,344-byte Solana bytecode format (SBF) binary. A finalized mainnet-beta transaction verified that proof, closed and refunded its proof account, advanced the pool sequence, and created the nullifier marker in one atomic transition. It consumed 1,344,003 of the 1,400,000 requested compute units. Program deployment, pool and proof-account setup, proof upload, and proof finalization were separate transactions. The security statements are argument soundness and conditional computational zero knowledge for the exact relation and declared view; they do not assert a statistical, perfect, or standard-model zero-knowledge result. Under an added round-by-round knowledge premise that posits a straight-line extractor — the knowledge analogue of the state-restoration premise, assumed rather than proved — the argument is an argument of knowledge, and, because the public nullifier binds the note to its spending secret, this yields conditional theft resistance against an adversary producing a spend in isolation: it knows the note's secret except with the stated soundness error. Theft resistance against an adversary that also observes honest spends needs a further non-malleability premise and is left open. The relation is a same-path leaf replacement: this release verifies a spend and does not include a deposit construction or a growing anonymity set.

1 Introduction

Transparent arguments remove the structured reference string and secret trapdoor required by many succinct proof systems. FRI and subsequent STARK constructions showed how low-degree testing and hash-based commitments can support transparent verification of large computations [4, 5]. Deploying such a verifier inside a blockchain transaction creates a second constraint: the verifier must fit the runtime's transaction budget while preserving the state-transition semantics of the application. Solana meters program execution in compute units and stores persistent application

state in explicitly addressed accounts [20, 21]. A proof that is inexpensive to verify on a workstation is therefore not, by itself, an on-chain construction.

This work studies one deliberately narrow application atom. A valid spend replaces one note by one output note at the same private path in a depth-20 tree, preserves the asset identifier, enforces value conservation including a public fee, derives a nullifier, and advances a public pool anchor and sequence number. The program accepts only if the proof verifies against that public state. In the same transaction it creates the nullifier marker and changes the pool state. This follows the commitment-and-nullifier structure used by shielded payment systems [2], but it is not a general wallet protocol, multi-input transaction, or public append operation.

The central systems choice is to separate proof transport from proof use. A proof is first written to a program-owned account and finalized. A later instruction reads the finalized bytes, verifies the complete proof, and performs the state mutation atomically. Thus, "one transaction" throughout this paper means the verification and state update from a finalized, pre-uploaded proof account. It does not include account creation, upload, or finalization. The separation keeps proof bytes available to the verifier without implying that the full proof lifecycle fits in one transaction.

Protocol. Aspis Spend uses arithmetic over $\mathbb{F}_{2^{31}-1}$ and a degree-four extension field. Its commitment layer is a custom, circle-domain, WHIR-style multilinear protocol. The term *WHIR-style* identifies the use of batched Reed–Solomon proximity testing, out-of-domain checks, recursive folding, and query openings in the lineage of WHIR [1]; it does not assert protocol equivalence. The frozen instance has rate $1/512$, $q = 18$ sampled query fibers, a batch-work parameter $g = 37$, two commitment layers, and four arity-four folds. A three-branch selector chooses the least schedule satisfying the exact $\mathsf{Good}_{\mathrm{spend}}$ rank conditions. The transcript, selector rule, serialization, and boundary count are part of the protocol definition rather than prover heuristics.

Security. The soundness argument is stated in the random-oracle model (ROM) and begins with an explicit interactive event ledger. It specializes proven Reed–Solomon proximity and mutual-correlated-agreement results to the circle code, batching map, folds, out-of-domain checks, and query schedule used by the implementation. It then applies a 32-boundary Fiat–Shamir accounting rule for a classical random-oracle adversary. For $1 \le T \le 2^{128}$ random-oracle queries, the released metric is false acceptance probability divided by T; after the conservative whole-ledger three-selector union, its minimum floor is 100.161 bits. The claim is argument soundness for the exact relation. No extractor or knowledge soundness claim is made.

The hiding statement is likewise tied to a precise experiment. It compares the complete declared verifier-visible Proof-or-Abort view with an explicit simulator in a programmable SHA-256 random oracle. The view includes the variable-length proof bytes, opened records, transcript and work values, selector or Abort result, serialization, program logs, and deterministic public mutation. It excludes network metadata, payer linkage, timing, local worker state, and physical side channels. With at most 2^{128} adaptive oracle queries, including post-output queries, and at most 17 proving attempts, the real-versus-simulator floor is 104.024 bits. The triangle inequality gives a 103.024-bit bound between the views of any two valid witnesses for the same public statement. This is a computational hiding result in the stated model, not statistical honest-verifier zero knowledge or a standard-model theorem.

Implementation and evidence. The verifier is implemented as a Solana SBF program. Its proof account has an application-level state machine: bytes may be uploaded before finalization, whereas the spend instruction requires the finalized state. Verification precedes every state write, and the

pool pre-state, sequence number, anchor, nullifier address, account ownership, and proof finalization state are checked at the instruction boundary. The frozen release binds a 65,407-byte proof to a 924,344-byte SBF binary. The maximum measured local path consumes 1,344,057 compute units. A finalized mainnet-beta execution of the exact release consumed 1,344,003 compute units, advanced the pool sequence from zero to one, created the canonical nullifier marker, and refunded the complete proof-account balance in the same transaction. Deployment, account creation, proof transport, and proof finalization occurred beforehand and are reported separately.

Contributions. The paper makes the following contributions.

1. It specifies the exact relation R_{spend}, public account pre-state, deterministic post-state, proof-account state machine, and byte-level transcript consumed by the program.
2. It gives a finite-parameter soundness derivation for the $q = 18$, rate-1/512 circle-domain instance, including the event union, selector loss, work schedule, and 32-boundary random-oracle reduction.
3. It constructs a witness-free simulator for the complete declared Proof-or-Abort view and states both the real-versus-simulator and pairwise witness-hiding games with their concrete bounds.
4. It connects the mathematical verifier to an SBF implementation and an atomic account transition, and reports a finalized mainnet-beta execution with the proof identity, deployed-program identity, instruction compute consumption, and setup lifecycle recorded separately.
5. It provides machine-readable ledgers, rank certificates, known-answer tests, proof and binary hashes, negative tests, and transaction evidence from which the quantitative claims can be regenerated.

Organization. Section 2 introduces the coding, transcript, and Solana concepts used in the construction. Section 3 defines the statement, relation, and account model. Section 4 specifies the protocol and wire format. Sections 5 and 6 prove soundness and computational hiding. Section 7 gives the implementation refinement and atomicity argument, and Section 8 reports the measurements and reproduction procedure. Section 9 states the scope of the result.

2 Background and related work

2.1 Shielded state transitions

A shielded payment system commonly represents an unspent note by a commitment inside an authenticated set and represents its consumption by a public, unique nullifier. A proof relates the nullifier and new commitment to a private opening and membership path without revealing the spent note. Zerocash gives an early complete treatment of this pattern in a decentralized payment system [2]. Aspis Spend retains only the cryptographic state-transition atom needed here: one input, one output, one asset, a public fee, and a same-path replacement in a depth-20 private tree. Wallet behavior, note discovery, transaction-origin privacy, and multi-party payment protocols are outside the relation.

It is useful to separate two statements that are sometimes conflated. The proof relation establishes that private note data are consistent with public inputs. The account transition establishes that the program consumed the expected public pre-state and wrote the specified post-state. The latter also depends on the host runtime's account-ownership, rollback, and locking rules; it is not a consequence of polynomial-commitment soundness.

2.2 IOPs of proximity and transparent arguments

An interactive oracle proof (IOP) lets a verifier query committed oracle messages instead of reading them in full [3]. The Fast Reed–Solomon IOP of Proximity (FRI) tests proximity to a Reed–Solomon code through recursive folding and a small number of authenticated queries [4]. This approach underlies transparent STARK systems [5]. DEEP-FRI strengthens the analysis by sampling outside the evaluation domain [6]. WHIR combines multilinear-polynomial evaluation with Reed–Solomon proximity testing and is designed for very fast verification [1].

The protocol in this paper belongs to that lineage but has its own transcript and proof format. It uses a circle-group evaluation domain, rate 1/512, two commitment layers, two out-of-domain/layer steps, four arity-four folds, and a degree-four final polynomial. Query positions address four-slot fibers. The transcript also includes a three-branch $\mathsf{Good}_{\mathrm{spend}}$ selector and explicit work nonces. Consequently, WHIR's parameters, proof grammar, and theorem statements are not imported by name; each lemma used here is specialized to the custom schedule.

2.3 Circle-domain Reed–Solomon codes

The base field is $\mathbb{F}_{2^{31}-1}$, often called M31. Its multiplicative group does not directly provide the large smooth domains used by conventional radix-two fast Fourier transform (FFT) constructions. Reed–Solomon codes over a norm-one circle group provide smooth evaluation domains in this setting [16]. Circle STARKs develop the corresponding arithmetization and proximity-testing machinery for Mersenne-prime fields [17]. The S-two construction provides a closely related concrete circle-code transcript and soundness development [10].

Aspis Spend uses the circle domain as a code domain while retaining a multilinear claim at the protocol boundary. The construction section defines the resulting ordering, fiber map, extension-field arithmetic, and fold maps explicitly. This distinction matters because an isomorphism of domains alone does not establish that a proximity theorem, batching argument, or query bound transfers to a different transcript.

2.4 Proximity gaps and correlated agreement

Soundness for a folded proximity protocol requires more than the distance of an individual Reed–Solomon word. The analysis must control lists of nearby codewords and show that correlated linear combinations do not allow inconsistent candidates to survive successive folds. Proximity-gap results formalize the first issue [7]. Mutual correlated agreement (MCA) formalizes the second. Recent results establish MCA preservation for polynomial generators and give Reed–Solomon MCA theorems in the Johnson regime [9, 15].

The distinction between a proved Johnson-regime statement and a conjectural capacity-regime statement is material. Crites and Stewart give counterexamples to Reed–Solomon proximity-gap conjectures used in earlier capacity-level analyses [12]. The Aspis Spend parameter proof therefore does not extrapolate a constant list bound to capacity. It instantiates only the proved proximity and MCA premises, then records every finite-field, out-of-domain, folding, query, hash, and selector failure event in a single ledger. The exact specialization is given in Sections 5 and 6.

2.5 Fiat–Shamir compilation and hiding

The implementation is non-interactive: verifier challenges and query indices are derived from a domain-separated SHA-256 transcript using the Fiat–Shamir transformation [13]. Security of this transformation is not captured by adding independent per-round errors. The BCS IOP framework

gives a random-oracle compilation and treats private oracle commitments [3]; later work analyzes Fiat–Shamir security for FRI and related SNARKs against bounded-query adversaries [8]. In Aspis Spend, the interactive event union and the random-oracle reduction are stated separately. The latter counts 32 prover-message boundaries and normalizes false-acceptance probability by the adversary's oracle-query work.

Hiding also depends on the complete transcript rather than only on the arithmetic trace. Full-field masks induce affine families of committed words, private Merkle salts hide unopened leaf preimages in the declared oracle model, and a simulator programs the transcript at fresh oracle inputs. The $\mathsf{Good}_{\mathrm{spend}}$ condition is designed to make the public affine image and mask fiber cardinality independent of the witness for every selectable schedule. The security theorem includes the selector, variable proof length, work nonces, serialization, program-visible logs, deterministic state mutation, and the possibility that all bounded attempts abort. This use of affine masking and constrained codewords is related to zero-knowledge IOPPs for constrained interleaved codes [11]; the concrete Aspis Spend simulator is specialized to its own fixed transcript and view.

2.6 Hash functions and concrete arithmetic

Arithmetic constraints and public note digests use Poseidon2 over M31; Poseidon2 is a sponge-friendly permutation designed for efficient algebraic implementations [14]. Commitments, transcript challenges, private-Merkle nodes, and work tests use domain-separated SHA-256. These uses have different roles. Poseidon2 collision resistance binds the public arithmetic digests, while SHA-256 is modeled as the programmable random oracle in the soundness and hiding games; SHA-256 itself is specified by FIPS 180-4 [18]. The implementation pins the concrete M31 Poseidon2 constants and arithmetic code to a specified Plonky3 revision [19]; the security section lists the corresponding assumptions rather than treating a software dependency as a theorem.

2.7 Solana execution and account state

Solana transactions invoke programs over an explicit account list. Accounts carry an owner and arbitrary program data, and only the owning program may modify that data under the runtime rules [20]. Programs are metered in compute units drawn from a transaction-wide budget [21]. A transaction either commits its account changes or rolls them back on error [24, 25], which makes verify-before-write control flow suitable for an atomic proof-authorized transition.

Large proof transport is nevertheless distinct from verification. Aspis Spend stores proof bytes in a program-owned account through a sequence of setup transactions, then changes the account to an application-finalized state. A spend instruction refuses an unfinalized account and does not expose an upload operation after finalization. "Finalized" here describes this application-level state machine; it is not a claim that account data are intrinsically immutable at the protocol layer. Likewise, a deployed SBF program remains upgradeable while its loader-v3 upgrade authority is present [22]. Deployment identity and authority state are therefore recorded separately from the cryptographic proof identity.

Network observations use Solana's commitment levels. In particular, "finalized" transaction evidence refers to a block recognized at the RPC interface's finalized commitment level [23], which is separate from the proof account's application-finalization bit.

3 Statement, relation, and state transition

This section fixes the object proved by Aspis Spend. The distinction between the arithmetic relation and the on-chain state preconditions is important: the proof claim is satisfiability of the former, while the accepting instruction checks the latter directly before changing any account.

3.1 Fields, digests, and domain-separated hashes

Let

$$p = 2^{31} - 1, \qquad \mathbb{F} = \mathbb{F}_p,$$

and fix the tower

$$\mathbb{F}_1 = \mathbb{F}[i]/(i^2 + 1), \qquad \mathbb{K} = \mathbb{F}_1[u]/(u^2 - (2 + i)).$$

Thus $|\mathbb{K}| = p^4$; the implementation denotes this fixed quadratic-over-quadratic representation by $\mathsf{QM31}$. A payment digest is an element of $\mathbb{F}^8$. All field elements have canonical little-endian encodings; an encoding of an integer greater than or equal to p is rejected.

The arithmetic relation uses a fixed width-16 Poseidon2 permutation over $\mathbb{F}$ [14]. We write H_{own}, H_{nul}, and H_{note} for the corresponding domain-separated sponge calls, and H_{node} for the width-16 two-to-one compression used by the atomic payment tree. The domains are part of the relation, rather than metadata supplied by a prover. In particular, an output note and a future input note use the same H_{note} domain. For digests $L, R \in \mathbb{F}^8$, the atomic node compressor places $L\|R$ in the permutation state, adds the fixed atomic-v3 tweak in the final lane, applies the permutation, and returns the first eight lanes.

For a bit b, define

$$\mathrm{Parent}_b(X, S) = \begin{cases} H_{\mathsf{node}}(X, S), & b = 0, \\ H_{\mathsf{node}}(S, X), & b = 1. \end{cases}$$

For $i = \sum_{j=0}^{19} i_j 2^j$ and $\boldsymbol{\sigma} = (\sigma_0, \ldots, \sigma_{19})$, set

$$\mathrm{Root}(L; i, \boldsymbol{\sigma}) = X_{20}, \quad X_0 = L, \quad X_{j+1} = \mathrm{Parent}_{i_j}(X_j, \sigma_j).$$

3.2 Public statement and witness

Definition 3.1 (Aspis Spend public statement)**.** A public statement is

$$x = (P, s, A, \nu, C_{\mathrm{out}}, A', a, f, \Delta),$$

where $P \in \{0,1\}^{256}$ is the pool account address, $s \in \{0, \ldots, 2^{64} - 1\}$ is its sequence number, $A, \nu, C_{\mathrm{out}}, A' \in \mathbb{F}^8$, $a \in \mathbb{F}$ is an asset identifier, $f \in \{0, \ldots, 2^{30} - 1\}$ is the public fee, and $\Delta \in \{0,1\}^{256}$ is the pool's deployment domain. The canonical 216-byte encoding is

$$\mathtt{version}\|\mathtt{depth}\|0^6\|P\|s_{\mathrm{le}}\|A\|\nu\|C_{\mathrm{out}}\|A'\|a_{\mathrm{le}}\|f_{\mathrm{le}}\|\Delta,$$

with version 4, depth 20, 32 bytes per digest, and four bytes for each of a and f. SHA-256 with the fixed statement domain hashes this encoding into the Fiat–Shamir transcript.

The deployment domain binds each statement to one program deployment on one named cluster. At pool initialization the program computes

$$\Delta = \text{SHA-256}(\texttt{aspis-spend-deployment-domain-v1} \,\|\, \mathrm{pid} \,\|\, \tau),$$

where pid is the runtime program id observed by the executing instruction and τ is a nonempty domain tag of at most 64 bytes (for example `mainnet-beta` or `devnet`) supplied on the initialization wire, and stores Δ in the pool account. Every state-affecting verification requires the statement's Δ to equal the pool's stored value before any other precondition involving proof bytes is evaluated. Because the statement digest is already absorbed into the Fiat–Shamir transcript, Δ is transcript-bound without a separate absorb step: a proof ground for one deployment domain cannot satisfy the transcript schedule of any statement carrying another.

Definition 3.2 (Aspis Spend witness)**.** A witness is

$$w = (k_\nu, r_{\text{in}}, r_{\text{out}}, pk_{\text{out}}, a_{\text{in}}, v, v', i, \sigma_0, \ldots, \sigma_{19}, \boldsymbol{\ell}_{\text{in}}, \boldsymbol{\ell}_{\text{out}}),$$

where each of k_ν, r_{in}, r_{out}, pk_{out}, and $\sigma_0, \ldots, \sigma_{19}$ lies in $\mathbb{F}^8$, $a_{\text{in}} \in \mathbb{F}$, $v, v' \in \{0, \ldots, 2^{30} - 1\}$, $i \in \{0, \ldots, 2^{20} - 1\}$, and each range vector belongs to $\{0, \ldots, 2^{10} - 1\}^3$.

Definition 3.3 (Atomic same-path relation)**.** The relation $R_{\text{spend}}(x, w) = 1$ holds exactly when all of the following equations hold over the indicated integer or field domain:

$$pk_{\text{in}} = H_{\mathsf{own}}(k_\nu), \tag{1}$$
$$L_{\text{in}} = H_{\mathsf{note}}(pk_{\text{in}}, v, a_{\text{in}}, r_{\text{in}}), \tag{2}$$
$$\nu = H_{\mathsf{nul}}(k_\nu, r_{\text{in}}), \tag{3}$$
$$C_{\text{out}} = H_{\mathsf{note}}(pk_{\text{out}}, v', a, r_{\text{out}}), \tag{4}$$
$$A = \text{Root}(L_{\text{in}}; i, \boldsymbol{\sigma}), \tag{5}$$
$$A' = \text{Root}(C_{\text{out}}; i, \boldsymbol{\sigma}), \tag{6}$$
$$a_{\text{in}} = a, \tag{7}$$
$$v' + f = v, \tag{integer}$$
$$v = \ell_{\text{in},0} + 2^{10}\ell_{\text{in},1} + 2^{20}\ell_{\text{in},2}, \tag{8}$$
$$v' = \ell_{\text{out},0} + 2^{10}\ell_{\text{out},1} + 2^{20}\ell_{\text{out},2}. \tag{9}$$

Each of the six limbs in (8)–(9) is proved to be a member of the exact 10-bit lookup table. Equation (integer) is an integer equality after the 30-bit range checks; it is not an equality modulo p.

The same i and the same twenty siblings occur in (5) and (6). Thus the relation replaces one private leaf along one private path. It does not prove a public append, a multi-input or multi-output transfer, or a statement about an otherwise general shielded pool. The fields P and s are transcript-bound public context. Their correspondence to live state is checked by the instruction, not inferred from the Merkle equations; in particular, s is a pool revision and is not the private path index i.

3.3 Trace realization

The arithmetic trace has $2^{10} = 1024$ rows. Its 49 Poseidon blocks comprise one owner-key block, three input-note sponge blocks, twenty input-path compressions, twenty output-path compressions, two nullifier sponge blocks, and three output-note sponge blocks. The two path computations share the physical direction-bit and sibling sources at every level. Tagged copy constraints enforce this sharing, while the Poseidon2, lookup, balance, public-binding, and terminal constraints enforce Equations (1)–(9). The generated atomic-v3 registry contains 183 copy terms and fixes the trace layout independently of the witness.

Proposition 3.4 (Relation realization)**.** *For every canonically encoded (x, w), the* Aspis Spend *trace builder produces an accepting arithmetic trace if and only if $R_{\text{spend}}(x, w) = 1$.*

Proof. The forward direction follows block by block. The owner, note, nullifier, and output-note blocks enforce the four domain-separated hash equations. At each Merkle level, the tagged two-item selection relation enforces the ordered pair (X_j, σ_j) or (σ_j, X_j) according to a Boolean copy of i_j; copy aliases force the input path, output path, and sibling row to use the same bit and sibling. The two terminal root bindings yield (5) and (6). Lookup membership and the two reconstruction constraints give the 30-bit ranges, after which the balance constraint is the integer equality (integer). The remaining public terminals enforce (3), (4), and (7).

Conversely, given a witness satisfying the displayed equations, fill each hash block with the deterministic Poseidon2 execution, place the common path bits and siblings in their registry cells, and use the stated 10-bit limbs. Every local, copy, lookup, and terminal constraint then vanishes. This argument concerns the algebraic trace definition; the polynomial argument that binds a proof to such a trace is given in Section 5. □

3.4 Live-state preconditions and atomic effect

For an accepting state-changing instruction, the verifier additionally requires the following public pre-state: (i) the supplied pool account has the expected owner, type, 80-byte length, key P, sequence $s < 2^{64} - 1$, anchor A, and stored deployment domain equal to the statement's Δ, with a distinct error code for a domain mismatch checked before the anchor comparison; (ii) the proof account has the expected owner and a finalized all-zero authority sentinel; and (iii) the nullifier marker is the canonical program-derived address with seeds `aspis-nullifier-v1` and ν, is unspent, and has one of the accepted pre-state shapes. Its 72-byte post-state stores and checks both P and ν. The instruction constructs x from these accounts and its public arguments, verifies the entire proof, reborrows and rechecks writable state, and only then commits

$$(s, A) \longmapsto (s + 1, A')$$

and the marker for (P, ν). A failure before the final commit leaves both objects unchanged. These account checks prevent a valid proof for one pool, revision, anchor, deployment domain, or nullifier from authorizing a different transition.

4 Aspis Spend construction

Aspis Spend is a transparent, WHIR-style polynomial argument over a circle evaluation domain. It uses the circle-code machinery developed for Circle STARKs [17] and an arity-four folding protocol in the lineage of FRI and WHIR [1, 4]. The transcript, oracle layout, masking system, and relation checks described below are specific to Aspis Spend; the construction is not an invocation of an unmodified WHIR parameter set.

4.1 Transparent parameters

Definition 4.1 (Transparent parameters)**.** The public parameters $\mathtt{pp}_{\text{spend}}$ consist of the field tower, Poseidon2 constants, circle-domain generators, fixed radix-four fold tables, the atomic-v3 constraint and copy registries, the 10-bit lookup table, Merkle domain tags, transcript labels, and the following shape.

These parameters require no structured reference string, trapdoor, or secret setup. They do require the fixed hash and random-oracle assumptions made explicit in Sections 5 and 6.

Table 1: Frozen Aspis Spend polynomial and transcript parameters.

quantity	value
trace/message rows	2^{10}
circle evaluation symbols	2^{19}
inverse rate	512
layer-zero query fibers	$N = 2^{17}$
queries per branch	$q = 18$, without replacement
generator width	29
generator order	semantic $0 \dots 15$, mask $16 \dots 25$, H_{26}, G_{27}, D_{28}
query branches per attempt	3
attempt cap	17
batch work	37 bits
fold work	$34, 33, 30, 25$ bits
final work	32 bits
folds	four arity-four folds
final polynomial	four coefficients in $\mathbb{K}$

4.2 Layer-zero oracles and batching

The layer-zero message contains 26 $\mathbb{F}$-valued codewords and three $\mathbb{K}$-valued codewords. The first sixteen codewords contain the randomized semantic trace. Ten additional full-domain codewords carry mask-only directions. The three extension-field lanes are denoted H, G, D. The D lane is a full 2^{10}-coefficient codeword with zero masking factor. It is nevertheless committed and opened as an ordinary lane.

The first commitment, C_1, stores the 26 subfield codewords. Four circle symbols form one 416-byte leaf. The second commitment, C_2, stores H, G, D; four extension-field symbols form one 192-byte leaf. After the nonzero batching challenge $\gamma \in \mathbb{K}^*$, the verifier combines the 29 lanes as

$$R_\gamma = \sum_{j=0}^{25} \gamma^j C_j + \gamma^{26} H + \gamma^{27} G + \gamma^{28} D. \tag{10}$$

The zero-factor lane is designed to add hiding directions without moving the committed batched word. For example, $\Delta G = -\gamma \Delta D$ implies $\gamma^{27} \Delta G + \gamma^{28} \Delta D = 0$. More generally, a source change U in lane $j < 27$ is paired with $\Delta G = -\gamma^{j-27} U$. The nonzero sampling of γ makes these expressions well defined. The complete source family and the legal root-neutral directions are checked by $\mathsf{Good}_{\mathrm{spend}}$, defined below.

The circle codeword has 2^{19} symbols, grouped into 2^{17} arity-four fibers. An arity-four fold produces the first line codeword; three more folds reduce through authenticated line layers of depths 15, 13, and 11 to a four-coefficient terminal polynomial. Each round contains two out-of-domain evaluations and a relation sumcheck. At a queried layer-zero fiber, the verifier decodes the C_1 and C_2 leaves, evaluates (10), and checks the first fold. It then checks the three authenticated line transitions and the terminal polynomial.

4.3 Fiat–Shamir schedule

The transcript uses SHA-256 with rigid, typed domain labels. Its logical order is as follows.

1. Absorb the profile header, field-basis identifier, canonical statement digest, masking context, and nonzero public mask nonce.
2. Absorb the C_1 root; sample λ and χ. Absorb the C_2 root and sample the zerocheck batching challenges.
3. Absorb the initial mask claim, sample nonzero η, verify and absorb the ten-round masked sumcheck, and derive its terminal point $z \in \mathbb{K}^{10}$.
4. Absorb the three statement points z, $\mathrm{succ}(z)$, and $\mathrm{xor}_{12}(z)$, their 84 extension-field evaluations, and the three D-lane evaluations.
5. Check and absorb the 37-bit batch-work nonce; sample $\gamma \in \mathbb{K}^*$ and the point scale.
6. For fold round $r = 0, \ldots, 3$, derive the two OOD points, absorb the two OOD values and relation sumcheck (and, for $r > 0$, the new layer root), then check and absorb the round's work nonce before sampling its fold challenge α_r.
7. Absorb the four-coefficient final polynomial; check and absorb the 32-bit final-work nonce.
8. Fork the resulting state into three branches. Branch b absorbs the single byte $b \in \{0, 1, 2\}$ under transcript label 44 and then samples 18 distinct query fibers.

The first seven steps are common to all three branches. In particular, the committed word, OOD checks, fold challenges, final polynomial, and all work nonces are fixed before the branch byte is absorbed. The three replays start from the same pre-branch state; they are not chained.

4.4 The complete $\mathsf{Good}_{\mathrm{spend}}$ predicate

Definition 4.2 (Public schedule)**.** A Aspis Spend schedule s_{fs} comprises the fixed layout identifiers, all public transcript challenges and OOD points, fold data, and one ordered 18-tuple of distinct query fibers. It excludes the witness, realized masks, private salts, committed roots as opaque inputs, opened values, and retry history.

The deterministic predicate $\mathsf{Good}_{\mathrm{spend}}(s_{\mathrm{fs}})$ reconstructs the public linear maps induced by this schedule and accepts precisely when all static pins, source guards, and rank conditions in Table 2 hold. Its source guards additionally check that each selected root-neutral source has zero compressed terminal contribution and zero pointwise γ-batched root difference.

The root-neutral basis is selected in increasing query-degree order: semantic and D sources precede mask-only sources. It contains 1068 degree-one and 336 degree-two query columns. Its total query degree is at most 31320. The root-neutral z-degree is at most 41040; the two terminal Schur blocks add 120 each, giving 41280. The γ-coordinate degree is at most 80688, and the complete continuous degree is therefore 121968.

Lemma 4.3 (Complete-Good algebraic liveness)**.** *For a uniformly sampled branch schedule, let B be the event that $\mathsf{Good}_{\mathrm{spend}}$ rejects. For three independent post-final branches from one common prefix,*

$$\Pr[B_0 \cap B_1 \cap B_2] \le \beta := \frac{121968}{p} + \left(\frac{31320}{131072 - 17}\right)^3 = 0.013705910239932412.$$

With at most 17 independent, completed attempts, the probability of an internal algebraic Abort caused by sampler/build failure or cap exhaustion is at most

$$17\frac{128}{p^6} + \beta^{17} < 2^{-105.21398677941983}.$$

Table 2: Rank conditions checked by $\mathsf{Good}_{\mathrm{spend}}$, over $\mathbb{F}$. "Schur" means rank after eliminating all query coordinates.

map	required rank	target dimension
root-neutral joint map	1404	1404
serialized terminal projection	324	324
terminal plus initial projection	328	328
$\ker(\text{initial}, T_z)$ coverage	1076	1076
remaining G/D query map	288	288
remaining G/D terminal Schur map	12	12
inactive-balanced H_1 query map	288	288
inactive-balanced H_1 terminal Schur map	12	12

Proof. Each nonzero determinant defining the complete product has continuous degree at most 121968, so Schwartz–Zippel bounds failure over its continuous challenges by $121968/p$. The exhaustive query-domain map has $N = 131072$ distinct roots and none equal to one. Sampling 18 roots without replacement therefore bounds failure of one query minor by $31320/(N - 17)$. The three label-44 branches are independent in the random-oracle experiment, producing the cube. Independent fresh attempts raise this bound to the seventeenth power; the first term accounts for the bounded six-coordinate sampler/build abort in each attempt. This is an internal algebraic availability bound, not the total probability of public Abort under a wall-clock controller. It does not include failure to complete an attempt before a deployment deadline. Soundness does not condition on the event in this lemma. □

4.5 Prover and verifier

One honest attempt proceeds in two phases. It first builds the common masked trace, the two layer-zero commitments, all OOD and fold data, the terminal polynomial, and the six canonical work records. It then evaluates $\mathsf{Good}_{\mathrm{spend}}$ on all three post-final schedules and selects the least good selector. Only the selected branch's five salted minimal-subtree opening sections are serialized. If all three branches are bad, the attempt is erased and repeated with fresh entropy and a new durably burned nonce. A non-rank construction or schema error is fatal but has the same public abort encoding. After 17 unsuccessful completed attempts the private builder records cap exhaustion.

The production executable accepts a caller-selected wall-clock boundary and uses 3,600 seconds when none is supplied. At the selected boundary its controller emits a payload-free Abort if no eligible proof has been recorded, whether because of an internal algebraic failure or because computation remains incomplete. This deadline event is not part of the probability in Lemma 4.3.

The verifier parses one canonical prefix, reconstructs the statement digest and the entire transcript, checks every work predicate, derives the selected 18 queries, authenticates the five opening sections, verifies the terminal and relation equations, and checks every fold transition. It need not run $\mathsf{Good}_{\mathrm{spend}}$: that predicate governs the honest prover's privacy and availability policy, not acceptance of adversarial proofs.

The state-changing instruction consumes these bytes from a finalized, pre-uploaded proof account. Proof-account creation, chunk upload, and finalization precede the accepting instruction. "One transaction" in this work refers to complete verification together with the nullifier and pool mutation, not to that setup lifecycle.

5 Argument soundness

This section gives a concrete classical random-oracle argument-soundness bound for the relation in Definition 3.3. The finite-parameter code reduction and the state-restoration property needed by the Fiat–Shamir compiler are stated explicitly as premises below. Section 5.6 states, under an added round-by-round knowledge premise, the extractor that upgrades this to an argument of knowledge and the theft-resistance corollary that follows.

5.1 Security experiment and metric

Let Π_{spend} denote the interactive public-coin protocol obtained by replacing each Fiat–Shamir challenge with an independent verifier message. The adversary may choose the public pre-state and statement x, receive the verifier messages adaptively, and return all oracle messages and openings. It wins if the verifier accepts while $\{w : R_{\mathrm{spend}}(x, w) = 1\} = \varnothing$. Account-state rejection can only reduce this event, so the algebraic analysis may conservatively omit the direct pre-state checks.

Remark 5.1 (Statement-encoding change). The v4 statement encoding extends the statement-digest preimage by the 32-byte deployment domain of Definition 3.1. The digest remains one 32-byte SHA-256 value absorbed at the same transcript position, so the Fiat–Shamir event ledger below is unchanged: no absorb boundary, challenge, or grinding predicate is added or moved by the new field.

For the Fiat–Shamir protocol, let a classical adversary make $T \in [1, 2^{128}]$ queries to the 256-bit random oracle, including queries made after it has produced candidate proof bytes. We report

$$\mathrm{Adv}^{\mathsf{snd}}_{\mathrm{spend}}(T) = \frac{\Pr[\text{the adversary produces an accepting false statement}]}{T}, \tag{11}$$

the success probability per random-oracle query. This normalization is appropriate for a protocol whose explicit grinding work is implemented by random-oracle search, and it is the metric used for the numerical claim.

5.2 Code transport and list binding

Write

$$\mathcal{C}' = \{p|_{G'_{19}} : p \in L'_{10}(\mathbb{K})\}.$$

This is the exact 1,024-coefficient subcode evaluated on the 2^{19}-symbol circle coset. S-two Theorem 5 and Equations (21)–(22) give $\dim L'_{10} = 2^{10}$ and $L'_{10} \subseteq L_{10}$. Its Definition 6, Lemma 1, and Section 1.4 identify the ambient L_{10} circle code, under the rational circle parametrization and a nonzero coordinate scaling, with a generalized Reed–Solomon code of degree at most 1,024; its Theorem 7 supplies the corresponding Johnson-regime list-decoding result [10]. Coordinate permutation and nonzero diagonal scaling preserve Hamming distance, agreement sets, and list sizes. Passing to $\mathcal{C}'$ can only shrink a decoding list.

Lemma 5.2 (Circle-to-GRS transport)**.** *For the 2^{10}-coefficient, 2^{19}-symbol Aspis Spend circle code, the implemented circle-to-line map identifies each relevant affine code slice with a scaled GRS slice having the same relative distance and the same agreement sets. The four implemented arity-four fiber folds commute with this identification.*

Proof. The circle parametrization pairs inverse points into the implemented arity-four fibers. Restricting a polynomial to the normalized circle and then applying $t = 2x^2 - 1$ gives the line evaluation

used by the verifier. On the selected coset the 131,072 fiber roots are distinct and none is one; the release evaluator checks all of them. Evaluation therefore differs from the corresponding GRS evaluation only by the fixed order and nonzero normalizers. Expanding the implemented fold gives the degree-three powers generator $a_0 + \alpha a_1 + \alpha^2 a_2 + \alpha^3 a_3$; the coordinate divisions are the invertible local FFT change of basis. S-two Lemma 4 and WHIR Lemma 4.13/Theorem 4.20 then give fold/list commutation for this powers generator, while the terminal constraint transports the linear subcode condition [1, 10]. □

The soundness ledger uses proven Johnson list decoding and the polynomial-generator MCA regime, rather than a capacity conjecture. The required multicodeword-agreement bounds are the one-domain specialization of S-two Protocol 3, Theorem 19, and Lemmas 3–4. For the scalar-powers generator, the corresponding Reed–Solomon MCA bound is Bordage et al., Definition 9.1 and Lemma 9.3; their Theorem 9.2 extends the result to every polynomial generator [9, 10, 15].

Lemma 5.3 (Johnson/MCA batching)**.** *Suppose the polynomial-batching, OOD, and local algebraic bad events listed in Table 3 do not occur. Then the two layer-zero commitments and their γ-batch in Equation* (10) *determine one member of the transported rate-*1/512 *code list, and every accepted fold and terminal value belongs to the corresponding folded list.*

Proof. The width-29 scalar-power batch is a polynomial-generator family. Outside its collision event, the MCA theorem reduces simultaneous proximity of the committed lanes to proximity of their single γ-combination. The rate-1/512 parameters lie inside the published proven-Johnson regime. By Lemma 5.2, the bound transfers to the implemented circle code. Linear arity-four folding carries every candidate list element to the next list. The two OOD samples and relation sumcheck at each layer bind the claimed list member; their exceptional events are charged separately in the ledger. Finally, the four-coefficient terminal check fixes the last folded polynomial. □

Lemma 5.4 (Local algebraic binding)**.** *Conditioned on the code/list conclusion of Lemma 5.3, an accepting transcript either encodes a trace satisfying Proposition 3.4, or one of the local events in rows 5–14 of Table 3 occurs.*

Proof. The relation OOD mixers bind the batched semantic, copy, lookup, and mask identities. A nonzero γ separates the 29 generator lanes and the three statement points. Tagged-tuple compression, copy denominators, and the θ-batch are sound outside their displayed collision or pole events. The zerocheck equality point and ten degree-27 rounds bind the trace constraints, while the H_1 and nonzero-η rows bind the masked sumcheck. When these events are absent, all constraints and public terminal equalities hold, so Proposition 3.4 supplies a valid witness. □

The preceding proofs identify the intended reduction, but the generated calculator and regression fixtures are not a formal verification of every imported-theorem hypothesis. The concrete theorem therefore isolates the finite-parameter step rather than silently treating executable agreement as a proof.

Assumption 5.5 (Pinned finite-parameter reduction)**.** For the exact code, transcript, and verifier serialized by the frozen artifact, Lemmas 5.2– 5.4 hold with the event partition and bounds in Table 3. In particular, the premise covers the rate-1/512 Johnson-list and polynomial-generator MCA hypotheses, the two OOD samples, all four folds and their list sizes, the terminal condition, and the assignment of every semantic, copy, lookup, range, mask, and sumcheck constraint to rows 5–14 of the table. The premise is uniform over every false public statement and every reachable public transcript prefix.

5.3 Selector scope

The selector byte is absorbed only after both commitments, all OOD data, folds, the terminal polynomial, and the final work nonce have been fixed. Let B denote the union of all bad events determined before that byte, and let M_b be the final query-miss event for branch b.

Lemma 5.6 (Three-branch selector containment)**.** *For every possibly adversarial function that selects $b \in \{0, 1, 2\}$ after observing all three schedules,*

$$B \vee M_b \quad \Longrightarrow \quad B \vee M_0 \vee M_1 \vee M_2.$$

Consequently the three-branch construction changes only the final query-miss term by a factor of at most three; it does not replicate any common-prefix term.

Proof. The implication is pointwise and does not require independence between the selector and the schedules. The common-prefix transcript replay additionally checks equality of the prefix challenges, OOD points, mixers, fold challenges, and pre-final-grinding state across all branches. A union bound over M_0, M_1, M_2 completes the argument. □

The release theorem below takes the still more conservative course of multiplying the complete transformed ledger by three. This avoids relying on the tighter scope allocation in its headline value.

5.4 Interactive event ledger

Table 3 lists bounds 2^{-b_i} for the 16 separately named failure families. The final-query row shows both the unselected one-branch value used to form the conservative ledger and, in parentheses, its factor-three selector-local value. The four fold rows have already been unioned into one table entry.

For the query row, take Johnson multiplicity $m = 10$, $\rho = 1/512$, and

$$\alpha = \left(1 + \frac{1}{2m}\right)\sqrt{\rho}.$$

Among $N = 131072$ fibers, the agreement cap is $A = \lfloor \alpha N \rfloor = 6082$. Sampling 18 distinct fibers therefore has miss probability at most

$$\prod_{i=0}^{17} \frac{6082 - i}{131072 - i}.$$

The 32-bit final work predicate raises the corresponding one-branch row to 111.7687016344 bits.

Rows 1–14 are the algebraic-event partition in Assumption 5.5. Rows 15–16 are release-policy slack, not collision probabilities derived for an adversary making 2^{128} queries. Argument soundness for the exact relation treats the Poseidon2 gates as part of R_{spend}, while the theorem below treats $\mathcal{H}$ as an ideal random oracle. Omitting these two positive reserve terms would only strengthen the numerical bound. Collision resistance of Poseidon2 is instead needed when a note commitment is interpreted as a binding name for application data outside the exact relation.

Using the one-branch final-miss row and unioning all 16 events gives

$$\varepsilon^{(0)}_{\text{round}} = \sum_i 2^{-b_i} \le 2^{-106.79080600295417}. \tag{12}$$

Under Assumption 5.5, the complement of this union and Lemmas 5.3 and 5.4 imply that R_{spend} is satisfiable.

Table 3: Aspis Spend interactive soundness ledger. Entries are $-\log_2$ upper bounds on the named event or conservative policy reserve.

event	bits
width-29 polynomial batch	107.3160201144
four-fold union	108.9854322658
final q18 miss, one branch (three-branch local union)	111.7687016344 (110.1837391336)
two-sample OOD-list union	213.1000183949
24 relation OOD mixers	119.4150374966
nonzero γ and three-point batch	119.0931094017
inactive-copy γ claim	119.1926450753
atomic tuple compression	112.3968373178
atomic copy/range poles	111.7627900366
atomic θ collision	119.4150374966
zerocheck equality point	120.6780719024
zero-sum H_1 helper	123.9999999973
nonzero η for the original mask	123.9999999973
ten degree-27 zerocheck rounds	115.9231844003
Poseidon2 semantic-binding policy reserve	124
SHA-256 instantiation policy reserve	128

5.5 Fiat–Shamir instantiation

The BCS compiler is characterized by soundness against state-restoration attacks, which is stronger than ordinary interactive soundness [3, 8]. A state-restoration adversary may save the verifier and oracle state at a public-message boundary, restore a saved state, and request a fresh continuation; its win event is acceptance of a false statement on any resulting branch. Let $\varepsilon_{\mathrm{sr}}$ denote the maximum win probability in the exact 32-boundary experiment for the serialized transcript.

Assumption 5.7 (Boundary state restoration)**.** For every false statement, every reachable saved state at each of the 32 boundaries, and every classical state-restoration adversary, the conditional escape events are covered by the event partition in Assumption 5.5. Consequently

$$\varepsilon_{\mathrm{sr}} \le \varepsilon^{(0)}_{\mathrm{round}}.$$

This is a round-by-round premise; it is not inferred from standard interactive soundness.

We use the BCS Fiat–Shamir analysis for IOPs [3], in the explicit bounded-query form of S-two Theorem 22 [10]. Under Assumption 5.7, its instantiation with $R = 32$ message boundaries and $\lambda = 256$ gives

$$\varepsilon_{\mathrm{BCS}}(T) \le (T + R)\varepsilon^{(0)}_{\mathrm{round}} + \frac{3(T^2 + 1)}{2^{\lambda}}. \tag{13}$$

At the upper query budget, the right-hand side of Equation (13) exceeds one and is therefore vacuous as a bound on raw success probability. The released claim is only the explicitly defined work-normalized metric in Equation (11); it does not book the raw bound as a security floor. Dividing by T gives the metric in Equation (11):

$$b(T) := \frac{\varepsilon_{\mathrm{BCS}}(T)}{T} \le \left(1 + \frac{32}{T}\right)\varepsilon^{(0)}_{\mathrm{round}} + \frac{3(T + 1/T)}{2^{256}}. \tag{14}$$

At $T = 1$, the multiplier of the round error is therefore 33. The right-hand side is convex for positive T, so its maximum on $[1, 2^{128}]$ is attained at an endpoint. Direct evaluation yields

$$-\log_2 b(1) = 101.74641188359571, \qquad -\log_2 b(2^{128}) = 106.79080421773332.$$

Table 4: Raw false-acceptance advantage $3\varepsilon_{\mathrm{BCS}}(T)$ by query budget, against the uniform per-query floor.

Query budget T	$-\log_2$ raw advantage
2^{40}	65.21
2^{64}	41.21
2^{80}	25.21
2^{100}	5.21
$\gtrsim 2^{105.2}$	vacuous
per-query floor (all T)	100.161

Theorem 5.8 (Work-normalized classical-ROM soundness)**.** *Assume the pinned finite-parameter reduction (Assumption 5.5), the 32-boundary state-restoration property (Assumption 5.7), and SHA-256 modeled as a 256-bit random oracle. For every classical adversary making* $1 \le T \le 2^{128}$ *oracle queries,*

$$\mathrm{Adv}^{\mathsf{snd}}_{\mathrm{spend}}(T) \le 3b(T) \le 2^{-100.16144938287455}.$$

Thus Aspis Spend *has a conservative 100.161-bit argument-soundness floor in the work-normalized metric.*

The work-normalized floor is a per-query bound. The cumulative raw advantage at a fixed query budget is $3\varepsilon_{\mathrm{BCS}}(T)$; it grows with T and becomes vacuous as T approaches the round error, as every grinding-based argument does. Table 4 reports both so the metric is not misread as a raw 2^{-100} forgery probability at every budget.

Proof. Assumptions 5.5 and 5.7, together with Equation (12), bound the state-restoration experiment. Equation (13) then applies the 32-boundary Fiat–Shamir transform. Multiplication by three is a whole-ledger upper bound for the three possible post-final query schedules. Evaluating the two interval endpoints after this factor gives 100.16144938287455 and 105.20584171701216 bits, respectively; the former is the minimum. Every accepted false statement is included in this event by the preceding lemmas. □

5.6 Knowledge soundness and theft resistance

Theorem 5.8 is a statement about the *existence* of a satisfying witness. For a spend, the property a pool needs is stronger: only a party that knows a note's authorization secret should be able to produce an accepting transition that consumes it. We record the additional premise under which the argument becomes an argument of knowledge, and the theft-resistance statement that follows from it and from the relation's binding structure. Like the soundness premises, the knowledge premise is assumed, not proved.

Assumption 5.9 (State-restoration knowledge extraction)**.** There is an expected-polynomial-time extractor Ext with the following property. For the exact code, transcript, and verifier of the frozen artifact, and for every classical state-restoration prover P^* that from any reachable saved state at each of the 32 boundaries drives the interactive protocol to an accepting terminal with probability exceeding $\varepsilon^{(0)}_{\mathrm{round}}$, Ext, given P^* together with its transcript of random-oracle queries and answers, outputs codeword openings and constraint assignments that satisfy every pinned event of Table 3, except with probability at most $\varepsilon^{(0)}_{\mathrm{round}}$. This is the round-by-round knowledge analogue of Assumption 5.7; it is not inferred from Theorem 5.8. It is deliberately heavier than its

soundness sibling: where Assumption 5.7 is a scalar probability bound, this premise is existential and algorithmic — it posits both the extractor and its success — and so is close to the interactive knowledge-soundness conclusion itself. The theorem below performs only the Fiat–Shamir transport of that premise; it does not discharge it.

Theorem 5.10 (Conditional non-interactive argument of knowledge)**.** *Assume Assumptions 5.5, 5.7, and 5.9, and SHA-256 as a programmable random oracle. There is a straight-line extractor E such that for every prover that outputs an accepting proof π for a statement x while making $T \le 2^{128}$ oracle queries, E, given that prover's query–answer transcript, outputs w with $R_{\mathrm{spend}}(x, w) = 1$, except with probability at most $3b(T) \le 2^{-100.161}$.*

Proof. The BCS Fiat–Shamir transform of a state-restoration interactive oracle proof is an argument of knowledge in the random-oracle model [3]: the extractor recovers the Merkle-committed oracle messages from the prover's query transcript, and by Assumption 5.9 their openings at the sampled positions determine codeword openings and assignments satisfying the pinned constraints. The pinned encoding of Section 4 maps that interactive witness to a relation witness w for R_{spend}. Extraction is straight-line from the query transcript, so it uses no rewinding. Its failure event — a satisfiable statement is accepted but Ext fails to recover the openings — is logically distinct from the soundness escape event of Theorem 5.8, in which a false statement is accepted; Assumption 5.9 bounds it by the same $\varepsilon^{(0)}_{\mathrm{round}}$, so the $3b(T)$ value carries to the knowledge error by that premise rather than by the soundness theorem. □

Corollary 5.11 (Authorization possession and theft resistance)**.** *Consider a note created with secret k_ν and input randomness r_{in}, so that its public nullifier is $\nu = H_{\mathsf{nul}}(k_\nu, r_{\mathrm{in}})$ by Equation (3). The nullifier is the only witness-derived value pinned in the public statement, so it alone identifies the note being consumed. Let A be any probabilistic polynomial-time party that outputs an accepting spend for a statement carrying ν while making $T \le 2^{128}$ oracle queries. By Theorem 5.10, except with probability $3b(T)$, the extractor yields a witness whose secret k and randomness r' satisfy $H_{\mathsf{nul}}(k, r') = \nu$. Since (k_ν, r_{in}) is a fixed preimage of the public target ν, any $(k, r') \neq (k_\nu, r_{\mathrm{in}})$ with the same image is a second preimage; by the Poseidon2 semantic-binding reserve $\varepsilon_{\mathrm{cr}}$ of the soundness ledger, which upper-bounds second-preimage advantage, $(k, r') = (k_\nu, r_{\mathrm{in}})$ except with probability $\varepsilon_{\mathrm{cr}}$. The extracted witness therefore contains the note's secret k_ν, and, through Equations (1) and (2), its owner key and input note as well. Hence any efficient party that produces an accepting spend of the note knows its authorization secret, and no efficient party lacking k_ν produces an accepting spend except with probability at most $3b(T) + \varepsilon_{\mathrm{cr}}$.*

Remark 5.12 (Scope, and the deployed-pool gap). Corollary 5.11 is a standalone statement: it bounds an adversary that produces an accepting spend without being handed honest proofs. A deployed pool is stronger — the adversary also observes honest spends. This is not covered by the corollary as proved. The honest proofs are produced by the simulator of Theorem 6.6, which programs the oracle at points the adversary need never query; an adversary that reuses a simulated Merkle root or opening inside its own proof can leave the corresponding preimage out of its query transcript, on which the straight-line extractor fails. Ruling this out is the standard malleability obstruction, and closing it requires a non-malleability premise — a weak-unique-response or simulation-extractability property of the compiled argument — of the same character as, and no weaker than, Assumption 5.9. We do not establish that premise here, so theft resistance against an adversary with access to honest spends is stated as open, not proved. Even the standalone guarantee is conditional on Assumption 5.9, which is assumed rather than proved, and concerns the production of accepting proofs only: it does not address key management, secret-key leakage, or the local and network observables excluded from the declared view of Definition 6.1.

6 Complete-view computational hiding

The hiding statement compares witnesses for one fixed valid public statement. It includes the proof's variable-length serialization and the public abort event; it does not condition on successful proving.

6.1 Declared view and games

Definition 6.1 (Complete declared view)**.** For public input x, the declared output is either

$$\mathsf{Proof}(V) \qquad \text{or} \qquad \mathsf{Abort}.$$

The value V contains the complete finalized proof-account bytes, all commitment roots, opened $\mathsf{value}\|\mathsf{salt}$ records and minimal-subtree frontiers, Fiat–Shamir and work nonces, the branch selector, proof length, program-visible application-instruction framing and logs, and the deterministic public pool and nullifier transition. It does not include the fee-payer signature or identity, recent blockhash, transaction origin, account-graph linkage outside the statement, or network metadata. The proof length is allowed to depend on the public query schedule. The output occurs at one fixed public release boundary.

The cryptographic games below do not use elapsed prover time as a random variable. They assume that each permitted attempt completes before the release boundary. Equivalently, they may be composed with an external cutoff flag that is independent of the witness, masks, oracle answers, attempt state, and private execution time: the same flag is coupled in both games, and a set flag maps either output to Abort. The internal cap-exhaustion event and this external truncation are analytically distinct even though they have the same public encoding.

Let $\mathcal{H} : \{0,1\}^* \to \{0,1\}^{256}$ be one programmable random oracle. Rigid domain encodings separate its transcript, field expansion, salt expansion, Merkle, statement, and work uses. In the real experiment $\mathsf{Real}^{\mathcal{H}}_{\mathrm{spend}}(x,w)$, the honest prover uses fresh attempt secrets, chooses the least good of three schedules, retries at most 17 completed attempts, and emits Definition 6.1. The simulator experiment $\mathsf{Sim}^{\mathcal{H}}_{\mathrm{spend}}(x)$ has no witness and may program $\mathcal{H}$ at fresh inputs.

A classical distinguisher receives arbitrary auxiliary input, observes the output, and may make adaptive oracle queries both before and after that output. Its total budget is $Q_H \le 2^{128}$. Define

$$\epsilon_{\mathrm{RS}} = \sup_{x,w,\mathcal{D}} \left| \Pr[\mathcal{D}^{\mathcal{H}}(\mathsf{Real}^{\mathcal{H}}_{\mathrm{spend}}(x,w)) = 1] - \Pr[\mathcal{D}^{\mathcal{H}}(\mathsf{Sim}^{\mathcal{H}}_{\mathrm{spend}}(x)) = 1] \right|,$$

$$\epsilon_{\mathrm{pair}} = \sup_{x,w_0,w_1,\mathcal{D}} \left| \Pr[\mathcal{D}^{\mathcal{H}}(\mathsf{Real}^{\mathcal{H}}_{\mathrm{spend}}(x,w_0)) = 1] - \Pr[\mathcal{D}^{\mathcal{H}}(\mathsf{Real}^{\mathcal{H}}_{\mathrm{spend}}(x,w_1)) = 1] \right|,$$

where each quantified witness satisfies $R_{\mathrm{spend}}(x,w) = 1$. The programming experiment follows the explicitly programmable random-oracle (EPRO) compilation framework of Ben-Sasson, Chiesa, and Spooner [3], specialized below to the concrete Aspis Spend transcript and private-Merkle wire.

6.2 Affine field view

Split every $\mathbb{K}$ element into four $\mathbb{F}$ coordinates. Fix a public schedule s_{fs}, and quotient the verifier's exact public linear identities. The complete non-hash field view of an honest prover has the form

$$Y_{s_{\mathrm{fs}}}(w,R) = A_{s_{\mathrm{fs}}} R + b_{s_{\mathrm{fs}}}(w), \tag{15}$$

where $R \in \mathbb{F}^m$ consists of ideal independent mask coordinates and $A_{s_{\mathrm{fs}}}$ is a public matrix determined by the frozen layout and schedule. Let the public verifier identities be

$$L_{x,s_{\mathrm{fs}}} y = c_{x,s_{\mathrm{fs}}}, \qquad \mathcal{S}_{x,s_{\mathrm{fs}}} = \{y : L_{x,s_{\mathrm{fs}}} y = c_{x,s_{\mathrm{fs}}}\}, \quad \mathcal{W}_{s_{\mathrm{fs}}} = \ker L_{x,s_{\mathrm{fs}}}.$$

The homogeneous space is independent of the witness and, after public values are moved to the right-hand side, of x.

Lemma 6.2 (Uniform image and constant fibers)**.** *Let $A : \mathbb{F}^m \to \mathbb{F}^n$ be linear, with rank r and image V. Every $v \in V$ has exactly $|\ker A| = p^{m-r}$ preimages. Consequently, if R is uniform in $\mathbb{F}^m$, then AR is uniform in V. If, in addition, $b(w_1) - b(w_0) \in V$, then $AR + b(w_0)$ and $AR + b(w_1)$ are identically distributed.*

Proof. For any $v \in V$, choose r_v with $Ar_v = v$. The full preimage is the coset $r_v + \ker A$, hence has p^{m-r} elements by rank–nullity. It follows that $\Pr[AR = v] = p^{m-r}/p^m = p^{-r}$ for each of the p^r elements of V. Finally, $b(w_1) - b(w_0) \in V$ implies $b(w_0) + V = b(w_1) + V$, so both translated uniform distributions have the same support and mass function. □

Assumption 6.3 (Complete affine-image and rank coverage)**.** For the frozen Aspis Spend prover, verifier, layout, and serialization, let $Y_{s_{\mathrm{fs}}}$ include every non-hash field coordinate in the declared view. For every valid (x, w) and every schedule accepted by $\mathsf{Good}_{\mathrm{spend}}$, Equation (15) is an exact representation of that complete vector, with independent uniform mask coordinates R and no omitted witness-dependent coordinate. Moreover, the production matrices have all eight ranks in Table 2, satisfy their source guards, and their semantic, G/D, inactive-balanced H_1, terminal, and root-neutral blocks cover the full homogeneous verifier space. Consequently,

$$\operatorname{im} A_{s_{\mathrm{fs}}} = \mathcal{W}_{s_{\mathrm{fs}}}, \qquad \mathcal{S}_{x,s_{\mathrm{fs}}} = y_{x,s_{\mathrm{fs}}} + \operatorname{im} A_{s_{\mathrm{fs}}} \tag{16}$$

for a canonical public origin $y_{x,s_{\mathrm{fs}}}$ computable by row reduction. In particular, for any two valid witnesses for x,

$$b_{s_{\mathrm{fs}}}(w_1) - b_{s_{\mathrm{fs}}}(w_0) \in \operatorname{im} A_{s_{\mathrm{fs}}}.$$

Writing $r_s = \dim \mathcal{W}_{s_{\mathrm{fs}}}$, the equality also gives $\operatorname{rank} A_{s_{\mathrm{fs}}} = r_s$ and $|\ker A_{s_{\mathrm{fs}}}| = p^{m-r_s}$.

The release supplies executable evidence for this premise. Every reconstructed mask direction satisfies the homogeneous public identities, so $\operatorname{im} A_{s_{\mathrm{fs}}} \subseteq \mathcal{W}_{s_{\mathrm{fs}}}$. For the reverse-coverage check, it decomposes a homogeneous field view into (i) the separately authenticated semantic raw coordinates, (ii) the physical H_1, G, D raw coordinates and their three terminal values, and (iii) the masked-sumcheck coordinates after fixing the initial and terminal claims.

The structural semantic maps cover every queried raw semantic coordinate; their balanced kernels and lane-separated terminal projections cover the remaining semantic terminals. The two 288-rank query maps and 12-rank Schur complements in Table 2 cover the remaining G/D and inactive-balanced H_1 raw coordinates. After these raw values are fixed, the remaining sumcheck difference belongs to $\ker(\text{initial}, T_z)$. The root-neutral product has rank 1404, of which 328 coordinates cover the terminal-plus-initial projection and 1076 cover that kernel. Hence every homogeneous solution is a sum of legal mask directions in the reconstructed system, yielding $\mathcal{W}_{s_{\mathrm{fs}}} \subseteq \operatorname{im} A_{s_{\mathrm{fs}}}$.

The same decomposition gives a constructive translation. Translate the semantic one-time pads to match their raw view; use inactive-balanced H_1 padding for the physical H_1 difference; pair D and source changes with G so that $\Delta G = -\gamma \Delta D$ and the batched word is unchanged; then use the 1076 root-neutral directions to cancel the remaining sumcheck difference. The combined message is now identically zero, so the linear encoder, OOD maps, folds, later queried values, and final polynomial also agree.

Finally, row reduction of the public affine identities selects a canonical solution by setting free coordinates to zero. This computes $y_{x,s_{\mathrm{fs}}}$ without a witness and checks the second equality in (16).

The rank equalities used in this coverage argument are finite, machine-checked statements over $\mathbb{F}$. The executable checker reconstructs the matrices from the frozen layout and public schedule,

performs elimination, verifies target dimensions, source guards, pivots, and nonzero minors, and rejects definition drift. The published component fingerprints record provenance; they are not substituted for matrix reconstruction. The certificate fixture supplies frozen witnesses for the nonzero minors, while the production transfer uses layout and $\mathsf{Good}_{\text{spend}}$-definition fingerprints and a fresh replay of all three public schedules. It does not assume that production proof bytes equal the certificate fixture.

This evidence binds the premise to the release, but it is not an independent verification of the completeness of the reconstructed affine model. The hiding theorems therefore remain conditional on Assumption 6.3 until such a verifier is published.

By Lemma 6.2 and Assumption 6.3, uniform ideal masks give exactly the uniform distribution on $\mathcal{S}_{x,s_{\text{fs}}}$. The affine replacement therefore has error $\epsilon_{\text{aff}} = 0$ on every emitted schedule under the premise.

6.3 Witness-free simulator

The simulator uses Equation (16) directly.

$\mathsf{Sim}_{\text{spend}}(x)$**.** For each of at most 17 attempts, create a fresh common transcript state and three label-44 branches using lazy random-oracle programming. Evaluate the public predicate $\mathsf{Good}_{\text{spend}}$ on all three schedules. If none is good, continue; if the cap is exhausted, output Abort. Otherwise select the least good schedule s_{fs}. Reconstruct the public affine system, compute its canonical origin $y_{x,s_{\text{fs}}}$, and row-reduce $A_{s_{\text{fs}}}$ to a canonical basis $v_1, \ldots, v_r$ of its image. Sample $c_1, \ldots, c_r \leftarrow \mathbb{F}$ and set

$$y = y_{x,s_{\text{fs}}} + \sum_{j=1}^{r} c_j \, v_j.$$

Use y as the complete non-hash field view. Simulate the five salted Merkle trees and their minimal frontiers, program the remaining transcript states lazily, sample the six canonical-minimum work nonces with their exact conditional laws, serialize the selected variable-length proof, finalize its proof-account frame, and append the deterministic public execution view.

Every input to this algorithm is public. In particular, $\mathsf{Good}_{\text{spend}}$ consumes no witness, realized mask, private salt, or retry-derived hidden value.

Lemma 6.4 (Selection and cap-exhaustion law)**.** *Conditioned on no adversarial prequery of a programmed input and no programmed-state collision, the three branch schedules, their* $\mathsf{Good}_{\text{spend}}$ *bits, the least-good selector, and the all-bad bit have a witness-independent joint distribution. The same holds for the first good completed attempt and the internal cap-17* Abort *event. This event does not include truncation caused by a wall-clock deadline.*

Proof. The three typed label-44 inputs extend one common state by distinct bytes. At fresh random-oracle inputs their outputs are independent and uniform, conditioned on the common prefix. $\mathsf{Good}_{\text{spend}}$ is a deterministic function of the resulting public schedule, so the Good bits and least-good selector inherit this witness-independent law. Fresh attempt entropy and a fresh burned nonce make attempts independent. Iterating the same coupling through the first good attempt, or through all 17 failures, proves the claim. Since Abort is retained in the output space, no conditioning or division by a success probability occurs. □

Table 5: Conservative distinct programmable-oracle input inventory per attempt.

family	count
salted-leaf preimages over five trees	305152
hidden leaf-salt derivations	305152
hidden Merkle-node forest upper bound	305152
field expander, including D	53892
seed and attempt binding	8
distinct transcript inputs	637
total C	969993

6.4 Private-Merkle and EPRO hybrids

For each opened leaf, the simulator samples the public 32-byte salt and computes the typed leaf hash. It assigns independent uniform labels to the maximal unopened frontier subtrees and hashes upward to the public root. A bounded distinguisher detects this replacement only by querying a hidden preimage or a programmed state, or through a collision. The same lazy programming method handles mask and salt expansion, transcript challenges, and adaptive queries after output.

For a work predicate with g leading work bits, the honest miner publishes the least successful 64-bit nonce. The simulator samples the exact truncated-geometric minimum, answers earlier nonces as failures, the minimum as a success, and later queries lazily. Across the six work states, the hardest predicate has $g = 37$; failure of the 64-bit search space is at most $6\exp(-2^{27})$ per attempt.

The adjacent hybrids are:

1. H_0: the honest prover with real mask, salt, transcript, branch, retry, and work randomness;
2. H_1: replace the non-hash view by the exact affine-image sample;
3. H_2: replace hidden expander inputs and salted-leaf preimages, preserving all opened value–salt records;
4. H_3: sample unopened Merkle frontiers and program roots and transcript challenges at fresh typed inputs;
5. H_4: replace remaining hidden oracle states lazily, charge programmed-state collisions, and retain consistent post-output answers;
6. H_5: generate three schedules, Good bits, least-good selection, retries, and internal cap-exhaustion Abort using Lemma 6.4;
7. H_6: replace the six work searches by their exact canonical-minimum laws;
8. H_7: serialize the schedule-dependent proof and append proof- account finalization, logs, framing, and deterministic state change;
9. H_8: the output of $\mathsf{Sim}_{\mathrm{spend}}(x)$.

The first step is perfect under Assumption 6.3. The distribution changes in H_2–H_4 are covered by one distinct-input inventory rather than by counting repeated SHA-256 invocations.

The transcript row comprises 47 typed absorbs, 292 possible challenge squeeze inputs, 292 matching advance inputs, and six work-predicate inputs. It includes the dedicated D-claim absorb and all three branch absorbs. Replaying a common prefix does not create new distinct inputs.

Lemma 6.5 (EPRO complete-view bound)**.** *Under Assumption 6.3, let $A \le 17$ be the number of completed attempts. Assume each attempt begins with fresh independent 256-bit field entropy and leaf-salt seed and a fresh, durably burned public nonce, and that only the fixed-boundary output in Definition 6.1 is public. Assume also the ideal-completion or witness-independent external-truncation boundary model stated above. Then*

$$\epsilon_{\mathrm{RS}} \le \frac{ACQ_H}{2^{256}} + \frac{\binom{AC}{2}}{2^{256}} + 6A\exp(-2^{27}), \qquad C = 969993. \tag{17}$$

Proof. Before a program point is exposed, the probability that at most Q_H adaptive oracle queries contain any of the AC hidden inputs is at most $ACQ_H/2^{256}$. This argument includes queries made after earlier public messages because the still-hidden program points retain uniform 256-bit states. A union bound over pairs of programmed inputs gives $\binom{AC}{2}/2^{256}$ for internal or cross-family state collisions. Outside those events, lazy programming makes every opened Merkle record, frontier, root, transcript answer, and later oracle query identically distributed in adjacent hybrids. Lemma 6.4 handles the branch and retry law. The canonical-work coupling is exact unless one of the six 64-bit nonce spaces contains no solution; the hardest 37-bit predicate contributes at most $\exp(-2^{64-37})$, yielding the last term. Proof-account framing and account mutation are deterministic functions of the simulated proof, statement, and public account branch and therefore add no error. □

At $A = 17$ and $Q_H = 2^{128}$, the three terms in Equation (17) have respectively

$$104.02492234825198, \qquad 209.04984478399368, \qquad 193635243.91255844$$

bits. The first dominates; the sum has more than 104.0249223482519 bits at the displayed precision.

Theorem 6.6 (Real view versus simulation)**.** *Assume the complete affine-image and rank-coverage premise in Assumption 6.3. In the declared programmable SHA-256 random-oracle model with ideal completion before the fixed boundary, or with the witness-independent external truncation defined above, for $Q_H \le 2^{128}$, every valid (x, w), and arbitrary auxiliary input,*

$$\mathsf{Real}_{\mathrm{spend}}(x, w) \approx_{\epsilon_{\mathrm{RS}}} \mathsf{Sim}_{\mathrm{spend}}(x), \qquad -\log_2 \epsilon_{\mathrm{RS}} > 104.0249223482519.$$

Equivalently, a three-decimal conservative floor is 104.024 bits.

Proof. Compose Assumption 6.3, the hybrid sequence, and Lemma 6.5. The simulator is witness-free by its construction, and both experiments include the same Abort outcome rather than conditioning on success. An independent external cutoff is coupled identically and therefore adds no distinguishing advantage. □

Corollary 6.7 (Pairwise-witness hiding)**.** *Under the same affine-image premise and boundary model, for any fixed x and any w_0, w_1 satisfying $R_{\mathrm{spend}}(x, w_i) = 1$,*

$$\epsilon_{\mathrm{pair}} \le 2\epsilon_{\mathrm{RS}}, \qquad -\log_2 \epsilon_{\mathrm{pair}} > 103.0249223482519.$$

A three-decimal conservative pairwise floor is 103.024 bits.

Proof. Apply the triangle inequality along $\mathsf{Real}_{\mathrm{spend}}(x, w_0) \to \mathsf{Sim}_{\mathrm{spend}}(x) \to \mathsf{Real}_{\mathrm{spend}}(x, w_1)$. □

7 Implementation

The implementation is a Rust workspace containing the prover, the native verifier, the release evaluator, and a Solana SBF verifier program. The on-chain program uses a deliberately small production dispatcher; legacy and diagnostic entry points are not linked into the release binary. This section specifies the byte formats and state transition implemented by that binary.

Table 6: Canonical atomic-payment statement payload.

Offset	Bytes	Field
0	1	version = 4
1	1	private Merkle depth = 20
2	6	reserved, all zero
8	32	pool address P
40	8	sequence s
48	32	current anchor A
80	32	nullifier ν
112	32	output commitment C_{out}
144	32	output anchor A'
176	4	asset identifier a
180	4	fee f
184	32	deployment domain Δ

7.1 Canonical public statement

The atomic public statement is

$$x = (P, s, A, \nu, C_{\mathsf{out}}, A', a, f, \Delta),$$

where P is the pool account address, s is its sequence number, A and A' are the current and next anchors, ν is the nullifier, C_{out} is the output commitment, a is the asset identifier, f is the fee, and Δ is the deployment domain. Table 6 gives the complete 216-byte encoding. All multibyte integers are little endian. Each 32-byte digest is eight canonical M31 limbs, each encoded as a four-byte integer less than $2^{31} - 1$. The asset identifier is also required to be canonical in M31, and the fee is required to be less than 2^{30}.

The transcript binding is

$$d_x = \mathrm{SHA256}(\texttt{aspis/atomic} - \texttt{payment} - \texttt{statement/v4} \,\|\, \mathsf{encode}(x)).$$

On chain, P is taken from the supplied pool account and s from the decoded pool state. They are not accepted as caller-provided instruction fields. The caller-supplied Δ must equal the pool's stored deployment domain before verification proceeds. This ensures that the statement checked by the proof is the statement for the state account that the transaction will mutate.

7.2 Proof serialization

Aspis Spend has a fixed 6,785-byte prefix followed by five authenticated opening sections. The first 6,736 bytes retain the base state-only layout; Aspis Spend appends three evaluations of the zero-factor D lane and one selector byte. Table 7 is the normative prefix map.

Physical position and transcript position are intentionally distinct for the append-only extension. The three D-lane claims are read from offsets 6736–6783 but absorbed with the other statement claims, before batch work and the layer-combination challenge. The selector is absorbed after the final work nonce. The parser checks canonical field encodings and the complete profile header before replaying the transcript.

After the prefix, the proof contains the five sections in Table 8. For a section with value width w, the wire format is

$$n_{16} \;\|\; \left(\mathsf{value}_w \,\|\, \mathsf{salt}_{32}\right)^n \;\|\; m_{32} \;\|\; \mathsf{frontier}_{32m},$$

Table 7: Aspis Spend fixed proof prefix.

Byte range	Bytes	Contents
0–15	16	versioned proof header
16–47	32	mask nonce
48–79	32	first-phase commitment root
80–111	32	second-phase commitment root
112–127	16	initial mask claim
128–4607	4480	masked sumcheck
4608–5951	1344	84 statement evaluations in QM31
5952–6623	672	four fold-round records
6624–6687	64	final polynomial
6688–6695	8	final work nonce
6696–6727	32	four fold work nonces
6728–6735	8	batch work nonce
6736–6783	48	three D-lane claims in QM31
6784	1	query-branch selector

Table 8: Authenticated opening geometry. Widths exclude the 32-byte leaf salt.

Tree	Binary depth	Value width (bytes)
first phase, layer zero	17	416
second phase, layer zero	17	192
later layer 0	15	64
later layer 1	13	64
later layer 2	11	64

where n_{16} and m_{32} are little-endian counters. Query indices are derived from the transcript and are not serialized. The parser requires the records to match the sorted, de-duplicated derived indices, authenticates each $\mathsf{value} \,\|\, \mathsf{salt}$ record against its commitment root, and rejects any byte remaining after the fifth section.

The common transcript state is cloned into three post-final branches. Branch $b \in \{0, 1, 2\}$ absorbs its branch label, derives an 18-query schedule, and evaluates $\mathsf{Good}_{\mathrm{spend}}$. The prover and release evaluator serialize the least-index successful branch and only that branch's openings. The on-chain verifier checks the serialized branch completely; least-$\mathsf{Good}_{\mathrm{spend}}$ selection is additionally enforced when the concrete proof is admitted by the release evaluator.

7.3 Accounts and proof lifecycle

Solana accounts provide persistent byte arrays with explicit ownership and writable locking [20, 25]. Aspis Spend uses a proof account, a pool account, and a nullifier-marker account.

Proof account. The proof-account image is

$$\mathtt{ASPU} \,\|\, \mathsf{proof\_len}_{32} \,\|\, \mathsf{upload\_authority}_{32} \,\|\, \mathsf{proof}[\mathsf{proof\_len}].$$

The 40-byte header is followed immediately by the proof. Tag 0 initializes a zeroed, program-owned account; tag 1 writes an in-range chunk after checking the upload authority. The lifecycle deliberately permits authorized overwrites before sealing. It does not maintain a chunk bitmap, so the executor compares the complete uploaded account image with the expected image before invoking tag 62.

Tag 62 seals the account by replacing the authority with 32 zero bytes. Production verification requires this zero-authority sentinel, and subsequent upload or finalization calls fail the authority check.

For the 65,407-byte release proof, the 40-byte account header gives an exact account allocation of 65,447 bytes. The mainnet executor uses 960-byte chunks, so a fresh upload uses 69 transactions: 67 full chunks and one 127-byte final chunk. It submits at most 16 uploads before waiting for the window to reach finality. Excluding program deployment, the deterministic fresh-account schedule is therefore:

1. create the 48-byte pool account and initialize it with tag 63;
2. create the 65,447-byte proof account and initialize it with tag 0;
3. submit 69 tag 1 upload transactions; and
4. seal the proof account with tag 62.

Pool creation and initialization are separate transactions; proof creation and tag 0 initialization share one transaction. The resulting application setup has 73 transactions. The subsequent tag 65 verification-and-transition call is a separate transaction. Program deployment is also separate from this application setup.

Pool account. The 48-byte pool image is

$$\texttt{ASPS} \parallel 1 \parallel 0^3 \parallel s_{64} \parallel A_{32}.$$

Tag 63 accepts only a zeroed, program-owned, writable account that also signs the initialization transaction. It checks the anchor encoding and that $s+1$ does not overflow before writing the initial state.

Nullifier marker. The canonical marker address is the program-derived address with seeds

$$(\texttt{aspis} - \texttt{nullifier} - \texttt{v1}, \nu).$$

The pool address is intentionally not a PDA seed. It is recorded in the 72-byte marker image

$$\texttt{ASPN} \parallel 1 \parallel 0^3 \parallel P_{32} \parallel \nu_{32}.$$

Consequently one nullifier has one canonical marker address for the deployed program, while the marker contents retain the pool to which it was applied.

7.4 Production instruction surface

The release binary recognizes only the lifecycle and verification tags in Table 9. Tag 61 is retained as an explicit fixed failure for the former diagnostic mutation path; all other tags are rejected before account processing. Fixed-width fields are decoded directly, without linking the historical generic instruction dispatcher.

The 137-byte tag 65 payload consists of the tag byte, four 32-byte digests $(A, \nu, C_{\mathsf{out}}, A')$, a four-byte asset identifier, and a four-byte fee. Tag 60 has the same payload and a read-only proof-account ABI. The mainnet tag 65 transaction supplies the five accounts in Table 10 in exactly that order.

Table 9: Minimal Aspis Spend production instruction surface.

Tag	Data bytes	Operation
0	5	initialize proof account
1	$9 + n$	upload n-byte proof chunk
59	178	read-only production verification
60	137	verify and apply, retaining proof account
61	1	explicit fail-closed diagnostic tag
62	1	finalize proof account
63	41	initialize pool account
64	1	close a finalized proof account and refund rent
65	137	verify, apply, and refund proof account atomically

Table 10: Tag 65 account interface.

Index	Access	Account
0	signer, writable	finalized program-owned proof account
1	writable	program-owned pool account
2	writable	canonical nullifier PDA
3	signer, writable	system-owned fee payer
4	read only	executable System Program

7.5 Atomic tag 65 transition

The tag 65 implementation uses the following order.

1. It validates ownership, signer and writable flags, account distinctness, the canonical nullifier PDA, the current pool anchor, and sequence non-overflow. A program-owned marker must be exactly 72 zero bytes; alternatively, the canonical PDA may be an empty System Program-owned account.

2. It constructs the canonical statement using the actual pool address and decoded sequence, then computes the statement digest.

3. It loads the finalized proof and runs the complete production Aspis Spend verifier. No cross-program invocation (CPI) and no account-data write occurs before this verifier returns success.

4. If the marker account is absent, it is created at the canonical PDA. A prefunded empty PDA is topped up, allocated, and assigned as necessary. The alternative program-owned-zero path requires no System Program CPI.

5. It reacquires mutable borrows, decodes and rechecks both state preconditions, constructs both final account images, refunds the complete proof-account lamport balance to the payer, writes (P, ν) to the marker, and replaces (s, A) in the pool with $(s + 1, A')$. The proof account must sign, so this refund cannot consume an account whose key is not controlled by the transaction builder.

The final transaction contains two Compute Budget instructions—a 1,400,000-CU limit and a 262,144-byte heap request—followed by one tag 65 application instruction. Solana transaction rollback covers both direct writes, the proof refund, and the marker-creation CPI, while writable

Table 11: Concrete Aspis Spend release instance.

Item	Value
trace rows / code rate	2^{10} / 1/512
query fibers	$q = 18$
batch work	37 bits
fold work	$(34, 33, 30, 25)$ bits
final work	32 bits
selector candidates / selected	3 / 0
proving-attempt cap	17
release gates	37 / 37
conservative soundness floor	100.161449 bits
real-versus-simulator floor	104.024922 bits
pairwise-witness floor	103.024922 bits

locks on the pool and canonical marker serialize conflicting spends [21, 25]. A duplicate therefore observes either the existing marker or the advanced pool state and is rejected. Tag 60 remains available when retaining the proof account is required; the evaluated mainnet transition used tag 65.

The term *one transaction* refers precisely to this verification and state transition from a previously finalized proof account, including the proof-account refund performed by tag 65. Program deployment, pool initialization, proof-account creation, proof upload, and proof finalization are setup operations. The instruction updates commitment and nullifier state; it does not invoke a Solana Program Library (SPL) token program or transfer application assets.

8 Evaluation

The evaluation separates three objects that have different scopes. The release certificate binds the proof, statement, verifier binary, and concrete security calculations. Local validator runs measure both supported account paths and exercise adversarial cases. A mainnet-beta run records one finalized verification-and-state-transition transaction for that exact release. Deployment and application setup precede the measured transition; resource recovery follows it.

8.1 Release instance

The fail-closed release evaluator rebuilt the manifest-default SBF binary, verified the concrete proof with the production host verifier, reconstructed all three selector branches, reconciled the local compute ledger, and checked 37 required gates. All gates passed and the failed-gate set was empty. The release-authorizing soundness value is the conservative whole-ledger, three-branch floor of 100.16144938287457 bits. The sharper selector-scoped calculation, 101.38010539241552 bits, is retained as a diagnostic and is not used for the headline claim.

The content identities used throughout the evaluation are:

Proof. 65,407 bytes; SHA-256 `32eb419e0c5c3ef4fa2db0d32579e88f1207547d8fb010279efeb6c05981b529`.

Statement sidecar. 767 bytes; SHA-256 `c35f6152ce4ffbfa6edeba68b4a6d2738e19b1420181c9500ba73f0c717c8699`.

Canonical public-input digest. SHA-256 `f022136a00af841abf2ffcff08969847e9490d1cef9b4dc1ae48f978d58a2bd8`.

Table 12: Production-binary local compute consumption.

Marker path	Tag 59 CU	Tag 65 CU	Headroom
program-owned zero marker	1,306,464	1,341,724	58,276
System-owned marker creation	1,306,464	1,344,057	55,943

Production SBF. 924,344 bytes; SHA-256 `e289faf85fe4773880794d5d4356461bb9cb94077b68711f99348efe19707d7e`.

Release certificate. SHA-256 `6efecd8cc00a4e52b111fb3dfb8775ce018fae089d4e309652269faf2dd4b740`.

The statement has sequence zero, asset identifier 17, and fee 1. Each of the three reconstructed schedules met the required $\texttt{Good}_{\text{spend}}$ ranks, so selector zero was both serialized and the least accepted selector. The proof, statement, and SBF identities were required to agree across host acceptance, local mutation, release, and mainnet preflight artifacts. The execution-time preflight used certificate `d83f6df645a00f28090e78f8648974ec94afd35f4677f5d84525ba6af7a2688f`. The publication certificate was regenerated after correcting stale, non-authorizing release metadata in its soundness source, removing duplicated release snapshots from the proof-independent complete-Good and HVZK sources, replacing local absolute command paths in the acceptance and mutation sources with repository-relative paths, and correcting a q18 comment in the program manifest. The explanatory certificate note was updated to describe that non-authorizing metadata boundary. A machine diff is limited to that note, the generation timestamp, byte lengths and digests for the five corrected JSON sources, and the program-manifest digest. The proof, statement, SBF, security values, compute values, and all 37 gate outcomes are identical. The release bundle retains both certificates.

8.2 Local compute and functional tests

Local compute was measured by transaction simulation on `solana-test-validator 2.3.0` (Agave source identifier `a2e21dda`, feature set `3640012085`). The frozen SBF was built with `solana-cargo-build-sbf` version `2.3.0`, platform-tools `v1.48`, and its bundled Rust `1.84.1`; a build producing different bytes fails the release identity gate. Each transaction requested a 1,400,000-CU limit and a 262,144-byte heap. The read-only tag 59 baseline and both tag 65 paths used the same uninstrumented release binary. Table 12 reports literal transaction totals.

The local production matrix also admitted only the intended feature set and rejected every union with the workspace's insecure diagnostic and test features. It accepted the mined proof on tags 59 and 65, rejected a separately committed unmined proof, and submitted corrupted-proof transactions to check rollback. Both marker paths produced the expected pool and nullifier images; a duplicate was rejected without a second mutation. A two-signer race on the canonical System-owned marker path produced exactly one commit. On each path a second pool was initialized through the real tag 63 instruction with a different domain tag; running the valid mined proof against it landed as a failure with exactly the deployment-domain mismatch error and changed no account. Successful tag 65 runs closed the proof account and reconciled its complete lamport refund.

8.3 Mainnet-beta method

The mainnet run used the Solana genesis hash

5eykt4UsFv8P8NJdTREpY1vzqKqZKvdpKuc147dw2N9d.

A read-only finalized snapshot first required the disposable Program, ProgramData, pool, and nullifier addresses to be absent. The executor then deployed the exact release binary under program

GQPNqfYF17Nj2dGsf6Q2AtiouyM67YxFZPh9LxBk2Ye3

with loader-derived ProgramData address

GoXXs2juAjmpvcsX7iLdHsGJCwBwB3HHQwvYyfV3RuU.

The finalized deployment transaction was recorded at slot 433,219,270. The executor checked the deployed code hash, exact maximum data length, loader ownership, Program–ProgramData linkage, and the dedicated upgrade authority before application setup, immediately before verification simulation, and after verification finality.

Application setup comprised 73 transactions: pool creation and initialization, combined proof-account creation and tag 0 initialization, 69 960-byte-or-smaller upload transactions, and tag 62 finalization. Pool initialization stored the deployment domain ba43feb01d7d7f5ee3f57a6481b202066c83c6c3e76020a619c1611abbd08c8f, the SHA-256 of the domain separator, the runtime program identity, and the cluster tag. Uploads were submitted in five finality windows of at most 16 transactions. The complete 65,447-byte proof account image was compared with the expected image before sealing. Simulated post-finalization upload and second-finalization attempts were rejected. Program deployment and these application-setup transactions are not part of the one-transaction verification claim.

The executor constructed one tag 65 transaction with a fresh blockhash, simulated the exact signed bytes with signature verification and without blockhash replacement, and submitted those same bytes. It allowed no changed transaction after signing and used no identical-wire retry. Finalized account images, transaction metadata, program continuity, and the program-log proof digest were checked before the run was recorded as successful.

8.4 Finalized mainnet-beta verification transition

The tag 65 transaction finalized at slot 433,219,840 on 16 July 2026 at 06:26:33 UTC. Its signature is

3G1voggszvDMGi5PbGM1kuEMYKvh2TNMbH6hHHwndUdRQJNT7ehRFpQpksxLnx5tp2xkS5jGi359rVXk42sRPFcv.

The signed wire had SHA-256 bdb3ff5446ef45332214e990f18d7c5f77423088c1872b88784bc7592c20070e; its message had SHA-256 cbd3f8e6c8d70a58a1ff67bd1c0d3d21c512c6da9e39d056949a36a039ecccc1. Simulation and the finalized record both reported 1,344,003 CU. This is 96.0002% of the requested limit and leaves 55,997 CU. The transaction fee was 10,000 lamports and the metadata reported no error.

The program emitted the domain-separated proof-digest log aspis-proof-sha256-v1. Decoding its value gives exactly the released proof's SHA-256, 32eb419e0c5c3ef4fa2db0d32579e88f1207547d8fb010279efeb6c05981b529; the executor required this equality in both simulation and finalized logs. The pool at 3ZRYarQZcWJQgzNwLo1Eo7PDmier3rbW41L8ccAddCvb advanced from sequence zero to one and its data hash changed from f1a5f7ce79a5d9b88360f0c12d23d2b1bfb18e7d07491e7101eae5a51d8530ea to f123ccd4f7901c1109c696d9882c5c39e481db749ad37cc0494cd3e617a4e2bf. The previously absent canonical nullifier marker CFJ5fYPSi4Qn6okBCZS3tXFMw7Lsz4Jy9vEAAGU5kfSU was created with the expected 72-byte image.

The proof account held 456,402,000 lamports immediately before the transition and zero afterward. The payer increased by the full proof balance, less the 1,392,000 lamports used to fund the new

nullifier and the 10,000-lamport transaction fee. The evidence reconciles this equation from the finalized pre- and post-balances. Proof verification, proof-account refund, nullifier creation, and pool mutation therefore occurred in the same transaction; none of the preceding setup transactions is included in that statement.

After finality, a duplicate-spend simulation used a separately created and sealed replay-probe account. It returned the exact nullifier-already-spent error and left the pool, nullifier, and replay-probe images unchanged. The probe was then closed with tag 64 and its 1,176,240-lamport balance was refunded. These replay-probe transactions are post-transition tests, not part of the verification transaction.

8.5 Resource recovery and cost accounting

The deployment was deliberately disposable. After the execution evidence had been made read-only, a separate loader-v3 close transaction finalized at slot 433,220,251. It removed ProgramData and refunded 6,434,638,320 lamports; the exact refund equation and 10,000-lamport fee were reconciled from the landed transaction. The small loader Program account, pool, and nullifier marker remained. Closing ProgramData disabled the disposable deployment after the measured verification; it did not alter the already finalized transaction record.

A later transfer moved the remaining payer balance out in a separate finalized transaction, leaving a zero payer balance; this treasury cleanup is not a protocol result and its destination is outside the scope of the evaluation. Of the amount that funded the payer, 0.0037584 SOL was retained on-chain as rent in the Program, pool, and nullifier accounts, and the remainder covered aggregate payer transaction fees across deployment and every payer-funded setup and cleanup transaction. These figures exclude the inbound funding transaction fee and are not the fee of the single verification transition, which was 10,000 lamports.

8.6 Independent reconciliation and reproducibility

The source snapshot, frozen publication bundle, and paper are available under the immutable tag `spend-q18-g37-mainnet-v1`. From that checkout, the bundle's offline verifier is `release/aspis-spend-q18-g37-mainnet-v1/verify.sh`; it authenticates every bundled object and checks the certificate lineage, proof and statement identities, deployed ProgramData image, finalized transaction observations, and cost equations.

The public reconciliation file `results/spend/spend_mainnet_independent_rpc_reconciliation.json` has SHA-256 `dca6e0070c4d2ac0406cab2a45c0c5c768315920d81ff63b623830744243beaf`. It records the sanitized `getTransaction` observations from the official Solana mainnet endpoint and an independent PublicNode endpoint. Both report the same mainnet genesis, signature, slot, block time, error status, fee, compute consumption, balances, and log digest for the verification transaction. Both also agree on the deployment and ProgramData-close records, and on the post-close account state in which the ProgramData account is absent and the Program account remains. Queries for the verification signature on official devnet and testnet returned null, which disambiguates the cluster. This machine-readable reconciliation, rather than an explorer rendering, is the source for the network claims in this section.

The archival reproducer `tools/reconstruct_spend_mainnet_sbf.py` independently obtains the same finalized 1,072-signature buffer history and raw transaction bodies from both providers. It binds each of 1,070 loader writes to the pinned buffer and authority and reconstructs all 924,344 SBF bytes without gaps or overlaps. The recovered SHA-256 is `e289faf85fe4773880794d5d4356461bb9cb94077b68711f99348efe19707d7e`, equal to the SBF frozen by that run's release certificate.

Applying the loader-v3 ProgramData header gives `708c5e5e171942b7b93df4188a564681fe49e476902014e634734b04ad8ade83`, equal to the pre-close account-data digest. The same replay checks the deployment identities and decodes the landed 169-byte tag-65 instruction data (the 137-byte core payload with a pinned 32-byte deployment-domain suffix), account order, proof-digest log, and refund equation. Its frozen output is `results/spend/spend_mainnet_sbf_and_instruction_reconstruction.json`.

Every published object—the release certificate, the immutable execution and cleanup receipts, and the independent reconstruction and reconciliation—is pinned by the bundle's `manifest.json` and `SHA256SUMS`. The reconciliation and the receipts record all principal signatures, the rent refunds, and the final cost equations, and none contains key material.

The release certificate can be reevaluated from the repository root with:

```
NO_DNA=1 cargo run -q -p aspis-prover --example \
  spend_soundness_epro_ledger -- --calculation-only
NO_DNA=1 cargo run --release -p aspis-xtask -- \
  spend-release
```

The release command rebuilds the manifest-default SBF before comparing it with the production-only binary and evaluating all 37 gates. Reproduction requires the proof and statement bytes; their hashes are identifiers, not substitutes. The evaluator records its current generation time, so a later reevaluation reproduces the substantive fields and gate outcomes but not the frozen certificate's byte identity or SHA-256.

9 Scope and limitations

The results have a deliberately narrow interpretation. Aspis Spend proves the atomic-v3 relation in Definition 3.3: one input note is replaced by one output note along the same depth-20 private path. It does not provide multiple inputs or outputs, a public append, recursion, proof aggregation, a wallet protocol, a relayer, or a general private application interface. The pool address, sequence, old and new anchors, nullifier, output commitment, asset identifier, and fee are public.

Theorem 5.8 is classical random-oracle argument soundness in the work-normalized metric. It makes no quantum-random-oracle claim, and its concrete ledger includes a 124-bit Poseidon2 assumption and treats SHA-256 as a 256-bit random oracle. Knowledge soundness and the theft-resistance corollary (Section 5.6) are conditional on Assumption 5.9, a round-by-round knowledge premise that is existential and algorithmic and is assumed rather than proved; theft resistance against an adversary that also observes honest spends is stated open, pending a non-malleability premise (Remark 5.12). The hiding results use the stronger programmable-random-oracle experiment; they do not instantiate a standard-model pseudorandom-generator theorem or a statistical zero-knowledge theorem.

The hiding view is exactly Definition 6.1. The equality $\epsilon_{\text{side}} = 0$ is by definition of that fixed public Proof-or-Abort channel. Local filesystem access, the burned-nonce ledger, scheduler and process timing outside the release event, power, thermal and memory observations, and remote prover or miner traffic are not in the modeled view. An implementation that exposes any of those channels requires a new simulator and leakage bound; the present deployment model therefore keeps rejected attempts and mining local and private.

The hiding game assumes ideal completion before the public boundary, or an external truncation event independent of the witness and all hidden prover state. The implementation instead accepts a caller-selected deadline (3,600 seconds by default) and publishes Abort when no candidate is

ready. No wall time or selected boundary is recorded for the current release proof. The algebraic bound in Lemma 4.3 excludes the wall-clock event and therefore is not a bound on the total deployed Abort probability. Applying the hiding theorem to a concrete deadline requires either a witness-independent readiness argument or a padded scheduling mechanism.

The privacy result also does not cover fee-payer linkage, wallet or relayer identity, transaction origin, the account graph beyond the public statement, or network-layer metadata. Those are deployment privacy questions rather than witness-hiding properties of the proof transcript.

The accepting instruction verifies and mutates state in one transaction only after a proof account has been created, funded, uploaded, checked, and finalized. Those preceding transactions, their latency, fees, rent, and storage are part of system operation but not part of the one-transaction claim. The mainnet run used a disposable upgradeable deployment so that ProgramData rent could be recovered after evidence capture. The exact code and authority were checked through verification finality, after which a separate close transaction removed ProgramData. The recorded transaction is therefore execution evidence, not a continuously available mainnet service.

The statement binds the pool public key, pool sequence and anchors, the spend fields, and the pool's deployment domain $\Delta = \text{SHA-256}(\text{sep} \,\|\, \text{pid} \,\|\, \tau)$, where pid is the runtime program id and τ the domain tag stored at pool initialization. The release certificate separately binds the frozen SBF digest. A proof ground for one deployment domain is rejected by any pool storing another, with a distinct error code, before proof bytes are interpreted; cross-deployment replay against an honestly initialized pool is therefore excluded by the statement itself rather than by operational discipline. The residual is keyholder-shaped: the holder of the program-id keypair can redeploy the same program id on another cluster and initialize a pool there with the same domain tag, reproducing an identical Δ. The binding is nevertheless independently checkable: any observer can recompute Δ from the public program id and domain tag and confirm which deployment a statement commits to. Economically linked deployments under distinct program ids or distinct domain tags cannot accept each other's proofs.

The hiding theorems are conditional on the complete affine-image and rank-coverage premise in Assumption 6.3. The $\mathsf{Good}_{\text{spend}}$ checker supplies release-bound evidence by reconstructing and eliminating the relevant finite-field matrices from the frozen layout. The fingerprints and regression tests detect drift, but no independent verifier of the reconstruction and implementation binding is presently published; the executable evidence is not a formal proof-assistant development or an external security audit. The finalized mainnet execution establishes behavior for one exact proof, statement, binary, account pre-state, and transaction. It is not a throughput benchmark, an availability result, or an external audit, and it does not establish resistance to denial-of-service or transaction-ordering attacks. Proof generation for the released instance reached its configured publication boundary; the paper does not claim a representative distribution of prover latency or memory use.

10 Conclusion

Aspis Spend combines a narrowly defined private-state relation, a circle-domain transparent argument, and a Solana account transition whose proof verification and public-state mutation occur atomically. For the frozen (q=18,g=37) instance, the conservative classical-ROM analysis gives a 100.161-bit work-normalized argument-soundness floor under the stated finite-parameter and state-restoration premises. Under the affine-image premise, the declared programmable-ROM Proof-or-Abort view has a witness-free simulator with a 104.024-bit real-versus-simulator floor and a 103.024-bit pairwise-witness floor.

The released 65,407-byte proof verifies under a 924,344-byte SBF program. The maximum measured local tag 65 path is 1,344,057 CU. The exact release subsequently executed on mainnet-beta: transaction `3G1voggszvDMGi5PbGM1kuEMYKvh2TNMbH6hHHwndUdRQJNT7ehRFpQpksxLnx5tp2xkS5jGi359rVXk42sRPFcv` consumed 1,344,003 CU, advanced the pool sequence, created the canonical nullifier marker, and refunded the finalized proof account. Independent finalized RPC records agree on the transaction and post-state. Program deployment, proof transport, and finalization were separate setup operations; ProgramData recovery and the treasury sweep were separate cleanup operations.

The result establishes feasibility for this one-input, one-output same-path transition under the measured Solana limit. Broader transaction relations, lower prover latency, independent security review, and sustained deployment engineering remain separate work.

A Canonical encodings and wire format

A.1 Arithmetic hash domains

The Poseidon2 sponge domains for owner derivation, nullifiers, and notes are, respectively,

$$\texttt{0x41530001}, \qquad \texttt{0x41530002}, \qquad \texttt{0x41530003} \quad \text{in } \mathbb{F}.$$

The atomic-v3 Merkle compressor uses the width-16 state $L\|R$, adds `0x41531005` to lane 15, applies the fixed Poseidon2 permutation, and returns lanes 0 through 7. The statement digest is

$$\text{SHA256}(\texttt{aspis/atomic-payment-statement/v4}\|\texttt{payload}).$$

The digest is a fixed 32 bytes absorbed at one transcript position, so the deployment-domain field below extends the preimage without adding a Fiat–Shamir boundary.

A.2 Public statement payload

Table 13: Canonical atomic-v4 statement payload.

offset	bytes	field	encoding
0	1	version	4
1	1	tree depth	20
2	6	reserved	all zero
8	32	pool address P	raw public key
40	8	sequence s	little-endian $u64$
48	32	current anchor A	eight canonical M31 limbs
80	32	nullifier ν	eight canonical M31 limbs
112	32	output commitment C_{out}	eight canonical M31 limbs
144	32	output anchor A'	eight canonical M31 limbs
176	4	asset a	canonical little-endian M31
180	4	fee f	little-endian $u32 < 2^{30}$
184	32	deployment domain Δ	raw 32 bytes

Changing any field changes the statement digest. The six reserved bytes, version, and depth are checked rather than ignored. The deployment domain $\Delta = \text{SHA256}(\texttt{aspis-spend-deployment-domain-v1}\|\textit{program}$ is stored by the pool at initialization and compared before any proof byte is interpreted.

A.3 Proof prefix

The fixed portion of a Aspis Spend proof has 6785 bytes. Its physical order is listed in Table 14. The transcript absorbs fields in the logical order of Section 4; in particular, the batch and fold work records are logically positioned before dependent challenges even though the three work arrays occupy a compact suffix.

Table 14: Aspis Spend fixed proof prefix. End offsets are exclusive.

start	end	bytes	object
0	16	16	canonical v4-s2 header
16	48	32	public mask nonce
48	80	32	C_1 root
80	112	32	C_2 root
112	128	16	initial masked claim
128	4608	4480	ten-round masked sumcheck
4608	5952	1344	84 statement-point evaluations
5952	6096	144	round-0 OOD values and relation sumcheck
6096	6272	176	round-1 root, OOD values, and sumcheck
6272	6448	176	round-2 root, OOD values, and sumcheck
6448	6624	176	round-3 root, OOD values, and sumcheck
6624	6688	64	four final-polynomial coefficients
6688	6696	8	final-work nonce
6696	6728	32	four fold-work nonces
6728	6736	8	batch-work nonce
6736	6784	48	three D-lane claims
6784	6785	1	query selector in $\{0, 1, 2\}$

The header fixes the spend proof profile, $\log_2$ message size 10, blowup logarithm 9, 18 queries, four rounds, raw-fiber fold payloads, radix-four minimal-subtree Merkle mode, final polynomial logarithmic length 2, and the exact flag set. All field encodings in the prefix are canonical.

A.4 Private opening sections

The suffix consists of five sections in the order C_1, C_2, W_1, W_2, W_3. Their Merkle depths and fixed opened-value widths are

$$(17, 17, 15, 13, 11), \qquad (416, 192, 64, 64, 64) \text{ bytes.}$$

Each section is encoded as

$$\texttt{count}_{u16} \,\|\, (\texttt{value}_{\text{fixed}} \| \texttt{salt}_{32})^{\texttt{count}} \,\|\, \texttt{frontier\_count}_{u32} \,\|\, \texttt{frontier}_{32}^{\texttt{frontier\_count}}.$$

The transcript-derived indices are not serialized. The parser derives the layer-zero indices and their successive right shifts, requires their canonical order, authenticates every value–salt record and frontier, and rejects trailing bytes. Consequently proof length varies with the minimal frontier induced by the public schedule.

A.5 Proof-account frame

The exact account image before finalization is

$$\texttt{ASPU} \| \texttt{proof\_len}_{u32,\text{le}} \| \texttt{upload\_authority}_{32} \| \texttt{proof}.$$

Authorized chunk writes affect only the declared proof payload. The FinalizeProof instruction (wire tag 62) replaces bytes 8 through 39 with zero and preserves the payload. Production verification accepts only the all-zero sentinel. The atomic pool is initialized separately by wire tag 63. Wire tag 60 retains the proof account; the evaluated mainnet transition uses wire tag 65 to verify, mutate state, and refund the proof account atomically. Wire tag 64 closes a separately finalized proof account without applying a spend.

B Transcript boundary and imported-theorem checks

The soundness evaluator treats the transcript as 32 prover-message boundaries for the BCS transform. These boundaries are fixed by the parser; changing the prefix shape, sumcheck round count, OOD records, commitment rounds, or selector position invalidates the parameter fingerprint rather than silently changing R.

The imported list-decoding/MCA step is instantiated only after the following local checks.

1. The layer-zero domain contains 2^{19} circle symbols and its 2^{17} arity-four fibers map injectively to line roots under $t = 2x^2 - 1$; no root is one.
2. The message has 2^{10} coefficients, giving rate $1/512$, and the selected proximity threshold lies in the proven Johnson regime.
3. The width-29 γ-batch is a polynomial-generator family over the fixed extension field, with $\gamma \neq 0$.
4. The two commitment phases and four arity-four folds use the same coordinate scaling as the circle-to-GRS transport.
5. Queries are an ordered sample of 18 distinct fibers; the verifier and numerical ledger use without-replacement probabilities.
6. Two OOD samples and the relation sumcheck are present at every fold layer, and the final object has exactly four coefficients.

These checks supply the hypotheses of the polynomial-generator MCA and circle transport results used in Lemma 5.3 [9, 10].

C Complete soundness arithmetic

The four individual fold-event bounds are

$$109.52994269233842, \quad 111.22628180453175, \quad 112.85813508022261, \quad 113.84064801383485$$

bits. Their union is the 108.98543226575069-bit row in Table 3. For one query branch, replacing the selector-local final-miss row by 111.76870163436465 bits and unioning all sixteen rows gives

$$-\log_2 \varepsilon^{(0)}_{\mathrm{round}} = 106.79080600295417.$$

The endpoint reconstruction is

$$\begin{aligned} b(1) &= 33\,\varepsilon^{(0)}_{\mathrm{round}} + 6 \cdot 2^{-256}, \\ b(2^{128}) &= \left(1 + 2^{-123}\right) \varepsilon^{(0)}_{\mathrm{round}} + 3(2^{128} + 2^{-128})2^{-256}. \end{aligned}$$

This yields 101.74641188359571 and 106.79080421773332 bits. Applying the release's whole-ledger factor three subtracts $\log_2 3$ from each endpoint, giving

$$100.16144938287455 \quad \text{and} \quad 105.20584171701216$$

bits. The first is the published floor.

The selector-local containment in Lemma 5.6 gives a strictly sharper diagnostic calculation by applying the factor three only to the final query-miss row. It is not used to raise the theorem's conservative headline value.

D $\mathsf{Good}_{\mathrm{spend}}$ certificate structure

The complete product consists of the following three nonzero minors.

Table 15: $\mathsf{Good}_{\mathrm{spend}}$ provenance anchors.

component	rank	fingerprint
root-neutral joint minor	1404	`0xc1e23e3acec55fc9`
remaining G/D terminal Schur minor	12	`0x7ca257aca211584a`
inactive-balanced H_1 terminal Schur minor	12	`0x2b09c8e0f643b2bf`
bound product	—	`0xb1af7cc08b30847d`

The first minor is selected from 10610 candidate $\mathbb{F}$-sources in minimum query-degree order. It contains 1068 degree-one query columns and 336 degree-two columns. Its query, root-neutral z, and γ-coordinate degree bounds are 31320, 41040, and 80688. Each Schur minor contributes 120 to the z-degree, producing the complete bounds

$$d_q = 31320, \qquad d_z = 41280, \qquad d_\gamma = 80688, \qquad d_{\mathrm{cont}} = 121968.$$

For a candidate schedule, the runtime checker performs the following steps:

1. verify the static profile, layout, generator order, nonzero γ, query count, query distinctness, and degree schema;
2. reconstruct every source column and target coordinate from the public layout and schedule;
3. verify pointwise root-neutrality and compressed-terminal-zero source guards;
4. eliminate the raw query blocks, compute the two terminal Schur complements, and require the ranks in Table 2;
5. construct the root-neutral initial/terminal/kernel map, recompute its pivots and nonzero minor, and require rank 1404; and
6. bind the dynamic component reports into the complete degree/product report.

The frozen fingerprints are reproducibility anchors for one known set of pivots. Runtime elimination may select different full-rank minors and does not accept a supplied fingerprint in place of rank. Transfer to another proof instance depends on the common frozen layout and public schedule maps, not equality of witness or proof bytes.

E Complete EPRO arithmetic

The field expander count is

$$2(22850 + 4 \cdot 1024) = 53892.$$

The transcript cap is

$$292 = 67 \cdot 4 + 3 \cdot 8$$

possible squeeze inputs: 67 bounded extension-field challenges use at most four blocks each, and each of the three q18 branches uses at most eight query blocks. Accounting for a matching state advance at every squeeze gives

$$637 = 47 + 2 \cdot 292 + 6.$$

Thus the full per-attempt inventory is

$$C = 3 \cdot 305152 + 53892 + 8 + 637 = 969993.$$

At $A = 17$ and $Q_H = 2^{128}$,

$$\begin{aligned} -\log_2\left(\frac{ACQ_H}{2^{256}}\right) &= 104.02492234825198, \\ -\log_2\left(\frac{\binom{AC}{2}}{2^{256}}\right) &= 209.04984478399368, \\ -\log_2\left(6Ae^{-2^{27}}\right) &= 193635243.91255844. \end{aligned}$$

The pairwise game traverses the simulator twice and therefore loses exactly one bit under the triangle inequality.

F Security games at a glance

Table 16: Quantifiers and outputs of the three security statements.

game	quantified witnesses	output/view	bound
argument soundness	no witness for chosen x	accepting proof	100.161 bits per random-oracle query
real versus simulator	one valid w for fixed x	complete Proof/Abort	104.024 bits
pairwise hiding	valid w_0, w_1 for fixed x	complete Proof/Abort	103.024 bits

The hiding games allow arbitrary auxiliary input and adaptive post-output queries within $Q_H \leq 2^{128}$. The soundness game is classical and uses the same query ceiling. The exact environmental scope of these games is collected in Section 9.

References

[1] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors,

Advances in Cryptology–EUROCRYPT 2025, Part IV, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025. doi: 10.1007/978-3-031-91134-7_8. URL https://doi.org/10.1007/978-3-031-91134-7_8. Cryptology ePrint Archive, Report 2024/1586.

[2] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE, 2014. doi: 10.1109/SP.2014.36. URL https://doi.org/10.1109/SP.2014.36.

[3] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In Martin Hirt and Adam D. Smith, editors, *Theory of Cryptography—TCC 2016-B, Part II*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016. doi: 10.1007/978-3-662-53644-5_2. URL https://doi.org/10.1007/978-3-662-53644-5_2.

[4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.14. URL https://doi.org/10.4230/LIPIcs.ICALP.2018.14.

[5] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL https://eprint.iacr.org/2018/046.

[6] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, volume 151 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:32. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.ITCS.2020.5. URL https://doi.org/10.4230/LIPIcs.ITCS.2020.5.

[7] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. *Journal of the ACM*, 70(5):1–57, 2023. doi: 10.1145/3614423. URL https://doi.org/10.1145/3614423.

[8] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat–Shamir security of FRI and related SNARKs. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology–ASIACRYPT 2023, Part II*, volume 14439 of *Lecture Notes in Computer Science*, pages 3–40. Springer, 2023. doi: 10.1007/978-981-99-8724-5_1. URL https://doi.org/10.1007/978-981-99-8724-5_1.

[9] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. All polynomial generators preserve distance with mutual correlated agreement. Cryptology ePrint Archive, Report 2025/2051, 2025. URL https://eprint.iacr.org/2025/2051. Revision dated 19 May 2026; PDF SHA-256 23519c2d5d6541ee53e635b10c22d5f5964301b79a853d9394da267062e520a6; accessed 15 July 2026.

[10] Dan Carmon, Lior Goldberg, Ulrich Haböck, Leonardo Lerer, Ilya Lesokhin, Shahar Papini, and Shahar Samocha. S-two whitepaper. Cryptology ePrint Archive, Report 2026/532, 2026. URL https://eprint.iacr.org/2026/532. Revision dated 24 March 2026; PDF SHA-256 e3b0132ec598ca16835c1de3c85d0c8b07c41b5f063f1d88b5a9628c22252c3f; accessed 15 July 2026.

[11] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. Cryptology ePrint Archive, Report 2026/391, 2026. URL https://eprint.iacr.org/2026/391.

[12] Elizabeth Crites and Alistair Stewart. On Reed–Solomon proximity gaps conjectures. Cryptology ePrint Archive, Report 2025/2046, 2025. URL https://eprint.iacr.org/2025/2046.

[13] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology–CRYPTO '86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1987. doi: 10.1007/3-540-47721-7_12. URL https://doi.org/10.1007/3-540-47721-7_12.

[14] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology–AFRICACRYPT 2023*, volume 14064 of *Lecture Notes in Computer Science*, pages 177–203. Springer, 2023. doi: 10.1007/978-3-031-37679-5_8. URL https://doi.org/10.1007/978-3-031-37679-5_8.

[15] Ulrich Haböck. A note on mutual correlated agreement for Reed–Solomon codes. Cryptology ePrint Archive, Report 2025/2110, 2025. URL https://eprint.iacr.org/2025/2110.

[16] Ulrich Haböck, Daniel Lubarov, and Jacqueline Nabaglo. Reed–Solomon codes over the circle group. Cryptology ePrint Archive, Report 2023/824, 2023. URL https://eprint.iacr.org/2023/824.

[17] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Report 2024/278, 2024. URL https://eprint.iacr.org/2024/278.

[18] National Institute of Standards and Technology. Secure hash standard (SHS). Technical Report FIPS PUB 180-4, National Institute of Standards and Technology, 2015. URL https://doi.org/10.6028/NIST.FIPS.180-4.

[19] Plonky3 contributors. Plonky3 M31 and Poseidon2, version 0.6.1. Source code, commit c5c6b17763e5fd4c6f793d9c42ce768f388dff72, 2026. URL https://github.com/Plonky3/Plonky3/tree/c5c6b17763e5fd4c6f793d9c42ce768f388dff72. Accessed 2026-07-14.

[20] Solana Foundation. Accounts. Solana documentation, 2026. URL https://solana.com/docs/core/accounts. Accessed 2026-07-14.

[21] Solana Foundation. Compute budget. Solana documentation, 2026. URL https://solana.com/docs/core/fees/compute-budget. Accessed 2026-07-14.

[22] Solana Foundation. Program deployment. Solana documentation, 2026. URL https://solana.com/docs/core/programs/program-deployment. Accessed 2026-07-14.

[23] Solana Foundation. Solana RPC methods and documentation. Solana documentation, 2026. URL https://solana.com/docs/rpc. Accessed 2026-07-14.

[24] Solana Foundation. Transaction pipeline. Solana documentation, 2026. URL https://solana.com/docs/core/transactions/transaction-pipeline. Accessed 2026-07-14.

[25] Solana Foundation. Transactions. Solana documentation, 2026. URL https://solana.com/docs/core/transactions. Accessed 2026-07-14.